PES UNIVERSITY

Mini project report on

MediRecord – Intelligent Clinical Records & Symptom-Similarity Search System

Submitted in partial fulfilment of the requirements for the award of degree of

Bachelor of Technology

in

Computer Science & Engineering

UE24CS351A – DBMS Experiential Learning (Level 3 Project)

Submitted by:

Name	SRN
Rakshitha K B	PES2UG24CS918
Mythri B H	PES2UG24CS290

under the guidance of
Dr. Nagasundari S

PES University

AUG – DEC 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
FACULTY OF ENGINEERING
PES UNIVERSITY
(Established under Karnataka Act No. 16 of 2013)
Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)
Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

CERTIFICATE

This is to certify that the mini project entitled

MediRecord – Intelligent Clinical Records & Symptom-Similarity Search System

is a bonafide work carried out by

Name	**SRN**
Rakshitha K B	PES2UG24CS918
Mythri B H	PES2UG24CS290

In partial fulfilment for the completion of fifth semester DBMS Project (UE24CS351A) in the Program of Study – Bachelor of Technology in Computer Science and Engineering under rules and regulations of PES University, Bengaluru during the period AUG. 2026 – DEC. 2026. It is certified that all corrections / suggestions indicated for internal assessment have been incorporated in the report. The project has been approved as it satisfies the 5th semester academic requirements in respect of project work.

Signature
Dr. Nagasundari S
Course Instructor, DBMS

DECLARATION

We hereby declare that the DBMS Project entitled "MediRecord – Intelligent Clinical Records & Symptom-Similarity Search System" has been carried out by us under the guidance of Dr. Nagasundari S and submitted in partial fulfilment of the course requirements for the award of degree of Bachelor of Technology in Computer Science and Engineering of PES University, Bengaluru during the academic semester AUG – DEC 2026.

Name	**SRN**	**Signature**
Rakshitha K B	PES2UG24CS918	<Signature>
Mythri B H	PES2UG24CS290	<Signature>

ACKNOWLEDGEMENT

We would like to express our gratitude to Dr. Nagasundari S, Department of Computer Science and Engineering, PES University, for her continuous guidance, assistance, and encouragement throughout the development of this UE24CS351A – DBMS Project.

We take this opportunity to thank Dr. Sandesh B J, Professor and Chairperson, Department of Computer Science and Engineering, PES University, for all the knowledge and support we have received from the department.

We are deeply grateful to Prof. Jawahar Doreswamy, Chancellor – PES University, and Dr. Suryaprasad J, Vice-Chancellor, PES University, for providing us various opportunities and enlightenment every step of the way. Finally, this DBMS Project could not have been completed without the continual support and encouragement we have received from our family and friends.

ABSTRACT

Small and mid-sized clinics still depend on paper records, messaging apps and disconnected spreadsheets to manage patient visits, prescriptions and billing. As a result, doctors cannot quickly retrieve a patient's history, and cannot find earlier patients who presented with similar symptoms, because ordinary SQL searches handle free-text clinical notes poorly.

MediRecord is a role-based clinic records system that addresses this using two database paradigms. Structured data – users, doctors, patients, appointments, visits, prescriptions, bills and an audit trail – is stored in MySQL across eleven normalised tables, protected by primary/foreign keys, CHECK and UNIQUE constraints, four triggers, a stored procedure (generate_bill), two views, transactions and indexes. Unstructured clinical notes and symptom tags are stored in MongoDB, where a full-text index with relevance scoring powers a Symptom Search that returns similar past cases; the matching visits are then joined back to MySQL at application level using the visit identifier.

The system is delivered as a Next.js web application with three roles (Admin, Receptionist, Doctor), JWT sessions in httpOnly cookies, bcrypt-hashed passwords and server-side role enforcement on every API route. This report documents the problem, ER model, relational mapping, DDL, DML, queries, triggers/procedures, and the front end, together with the testing carried out.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1.	INTRODUCTION	10
2.	PROBLEM DEFINITION	11
3.	ER MODEL	12
4.	ER TO RELATIONAL MAPPING	14
5.	DDL STATEMENTS	16
6.	DML STATEMENTS	31
7.	QUERIES	36
8.	STORED PROCEDURES, FUNCTIONS AND TRIGGERS	40
9.	FRONT END DEVELOPMENT	49
10.	TESTING	54
11.	CONCLUSION	55
	REFERENCES / BIBLIOGRAPHY	56
	APPENDIX A DEFINITIONS, ACRONYMS AND ABBREVIATIONS	57

LIST OF TABLES

Table No.	Title	Page No.
1.1	Technology stack	10
2.1	Functional requirements	11
2.2	User roles and permissions	11
4.1	Relational schema (11 tables)	14
8.1	Triggers	42
9.1	Demo logins (seeded)	53
10.1	Test cases	54

LIST OF FIGURES

Figure No.	Title	Page No.
3.1	ER diagram (Chen notation)	12
3.2	ER diagram (relational notation)	13
4.1	Relational mapping of all 11 tables with keys	15
4.2	System architecture: browser, Next.js API, MySQL and MongoDB	15
5.1	Creating the database and the first tables	19
5.2	Tables created without errors	20
5.3	SHOW TABLES – all 11 tables present	21
5.4	DESCRIBE of core tables	22
5.5	Constraints and foreign keys (SHOW CREATE TABLE)	23
5.6	Index listing	24
5.7	SHOW TABLES: 11 tables and 2 views (13 rows)	25
5.8	DESCRIBE role, app_user and doctor	26
5.9	DESCRIBE patient	27
5.10	DESCRIBE visit and medicine	27
5.11	DESCRIBE prescription and prescription_item	28
5.12	DESCRIBE bill and audit_log	28
5.13	SHOW CREATE TABLE appointment: primary key, foreign keys (RESTRICT) and CHECK	29
5.14	ALTER TABLE: add and drop a column	29
5.15	CREATE TABLE temp_demo (13 → 14 objects)	30
5.16	DROP TABLE temp_demo (back to 13 objects)	30
6.1	Loading the demo data (seed-demo.js)	31
6.2	INSERT into patient and appointment inside a transaction	31
6.3	role and app_user tables	32
6.4	doctor table	32
6.5	patient table	32
6.6	medicine table	33
6.7	appointment table	33
6.8	visit table	33
6.9	prescription table	34
6.10	prescription_item table	34
6.11	bill table (generated by stored procedure)	34
6.12	Row count per table	35
7.1	Five-table JOIN with LEFT JOIN	36
7.2	GROUP BY with HAVING	36
7.3	Aggregate with ORDER BY	37
7.4	UPDATE inside a transaction	37
7.5	DELETE of never-prescribed medicines (rolled back: 12 medicines again)	38

Figure No.	Title	Page No.
7.6	Correlated subquery: bills above the average of the others	38
7.7	Nested query with IN	38
7.8	LEFT JOIN returns an empty set: every demo patient has an appointment	39
8.1	SHOW CREATE PROCEDURE generate_bill	41
8.2	Calling generate_bill inside a transaction: 500.00 + 0.00 + 25.00 tax = 525.00	41
8.3	After ROLLBACK the bill count is back to 8	41
8.4	SELECT from doctor_revenue_summary (revenue counts paid bills only)	42
8.5	The four triggers registered in the database	44
8.6	SHOW CREATE TRIGGER trg_visit_update	44
8.7	Trigger fires on UPDATE of visit: audit_log row 10 created (then rolled back)	44
8.8	audit_log contents after seeding	44
8.9	The seven statements the database rejects (CHECK, UNIQUE, FOREIGN KEY, RESTRICT)	45
8.10	Typed integrity violations in the MySQL client	45
8.11	EXPLAIN: the appointment lookup by doctor uses idx_appointment_doctor (type ref)	45
8.12	Passwords are bcrypt hashes (60 characters, prefix $2b$10$)	46
8.13	clinical_notes: 8 documents and its indexes (text index, unique visit_id, doctor_id, diagnosis_code)	46
8.14	Data model: one clinical_notes document	47
8.15	Ranked full-text search with $text and textScore	47
8.16	Filtering by diagnosis code J45	47
8.17	Aggregation: notes per doctor and symptoms per diagnosis	48
9.1	Home page with Sign in	49
9.2	Admin – analytics (revenue per doctor from the SQL view)	49
9.3	Admin – manage doctors	50
9.4	Receptionist – register patient	50
9.5	Receptionist – book appointment	51
9.6	Receptionist – generate bill (stored procedure) and bill list	51
9.7	Doctor – my appointments	52
9.8	Doctor – completed visit: vitals and diagnosis (MySQL), notes and symptoms (MongoDB), prescription and bill	52
9.9	Doctor – Symptom Search: ranked similar cases from MongoDB joined with SQL data	53

1. INTRODUCTION

Healthcare providers generate a continuous stream of data: patient demographics, appointments, vitals, diagnoses, prescriptions and bills. In many small and mid-sized clinics in India this information still lives in paper registers, WhatsApp messages or spreadsheets. Retrieving a complete patient history is slow, errors creep in, and there is no reliable audit trail.

MediRecord is a clinic information system that stores structured records in a relational database (MySQL) and free-text clinical notes in a document database (MongoDB). Doctors can record a visit, prescribe medicines and – most importantly – search past clinical notes by symptom to find similar cases, while receptionists manage registration, appointments and billing, and the administrator manages doctors and reviews revenue.

1.1 Objectives

- Digitise patient, appointment, visit, prescription and billing records in a normalised relational schema with enforced integrity.
- Store flexible, free-text clinical notes and symptom tags in MongoDB and provide ranked full-text symptom-similarity search.
- Demonstrate advanced SQL features: JOINs, GROUP BY/HAVING, subqueries, views, triggers, stored procedures, transactions and indexing.
- Provide secure, role-based access (Admin, Receptionist, Doctor) with hashed passwords and server-side authorization.
- Deliver an end-to-end working web application that integrates both databases.

1.2 Scope

In scope: user and role management, doctor and patient registration, appointment booking, visit recording (vitals and diagnosis in MySQL; notes and symptoms in MongoDB), prescriptions with multiple items, automatic billing, audit logging, an analytics dashboard and symptom-similarity search. Out of scope: online payment gateways, laboratory/imaging integration, patient-facing portals and e-prescription transmission.

1.3 Technology Stack

Table 1.1: Technology stack

Layer	Technology
Front end	Next.js 16 (App Router), React 19, Tailwind CSS v4
Back end	Next.js API routes (Node.js)
SQL database	MySQL 8.0 (mysql2 driver, connection pool)
NoSQL database	MongoDB (official Node driver) – collection clinical_notes
Security	JWT in httpOnly cookie (8 h), bcryptjs, route guard (proxy.js) + requireRole()
Version control	Git and GitHub

2. PROBLEM DEFINITION

Small and mid-sized clinics rely on paper records, WhatsApp messages or disconnected spreadsheets for patient visits, prescriptions and diagnostic history. Doctors and staff struggle to access complete patient histories quickly, and existing systems cannot find earlier patients with similar symptoms: a traditional SQL LIKE search over free text is slow, unranked and misses synonyms and word forms.

2.1 Functional Requirements

Table 2.1: Functional requirements

ID	Requirement
FR1	Role-based login for Admin, Receptionist and Doctor; pages and APIs restricted by role.
FR2	Admin creates doctor accounts (user + doctor profile atomically).
FR3	Receptionist registers patients.
FR4	Receptionist books appointments for a doctor at a date/time.
FR5	Doctor sees only his/her own appointments.
FR6	Doctor records a visit: vitals and diagnosis (MySQL) plus symptoms and notes (MongoDB).
FR7	Doctor adds a prescription with several medicine items in one transaction.
FR8	Receptionist generates a bill via stored procedure and marks it paid.
FR9	Doctor searches past notes by symptoms and sees similar cases joined with SQL data.
FR10	Admin views analytics: revenue per doctor, appointments, unique patients.

2.2 Roles

Table 2.2: User roles and permissions

Role	Permissions
Admin	Create doctors; view analytics (doctor_revenue_summary view); view billing.
Receptionist	Register patients; book appointments; generate and update bills.
Doctor	View own appointments; record visits; prescribe; symptom search; view completed visit details.

3. ER MODEL

The conceptual model identifies the entities Role, User, Doctor, Patient, Appointment, Visit, Medicine, Prescription, Prescription Item, Bill and Audit Log. A User has exactly one Role; a Doctor profile extends a User (1:1). A Patient has many Appointments; a Doctor has many Appointments; an Appointment results in at most one Visit; a Visit has at most one Prescription and at most one Bill; a Prescription has many Prescription Items, each referring to a Medicine. The Audit Log is written by database triggers. The clinical note itself is held in MongoDB and linked through Visit.mongo_note_id.

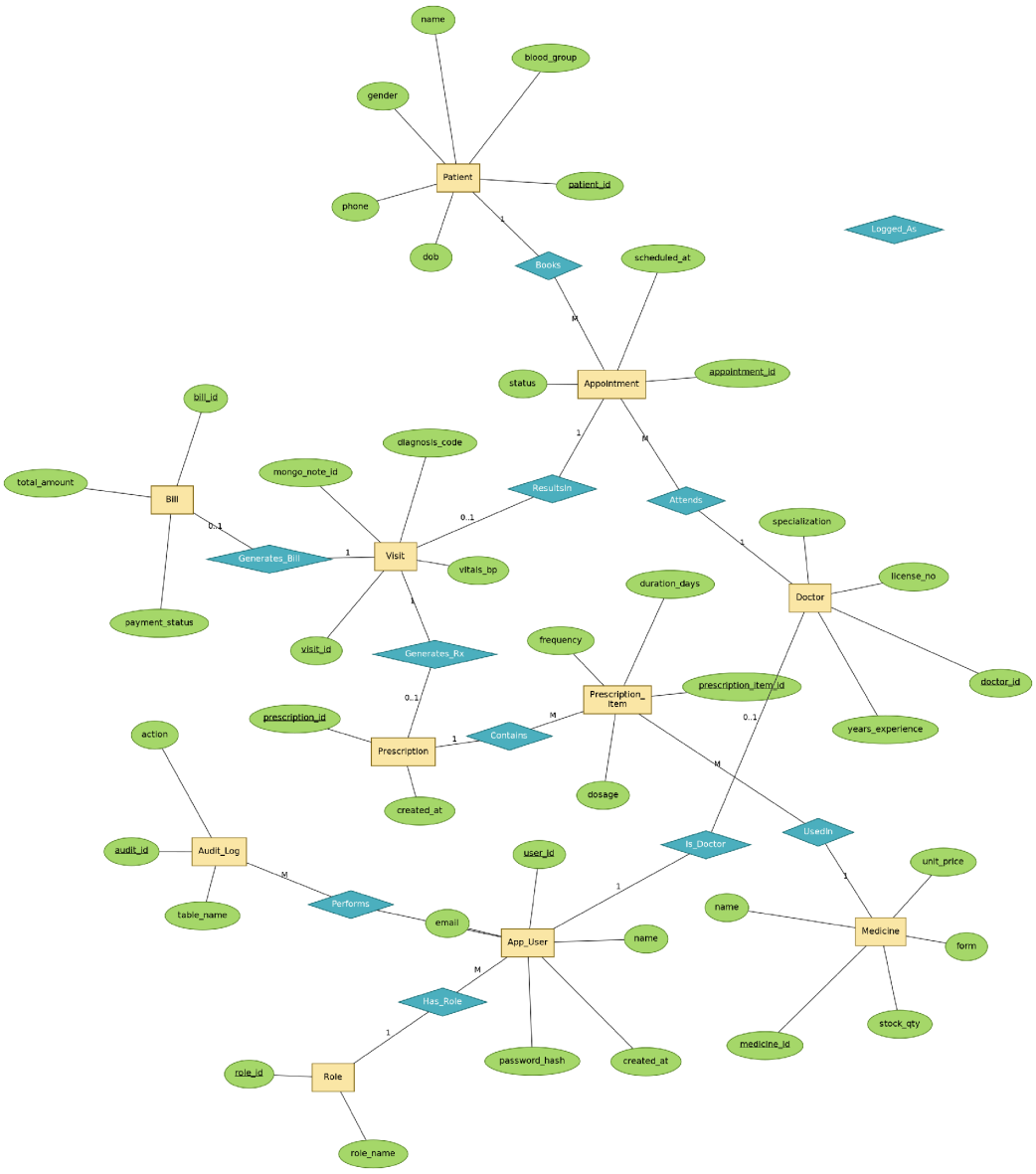


Figure 3.1: ER diagram (Chen notation)

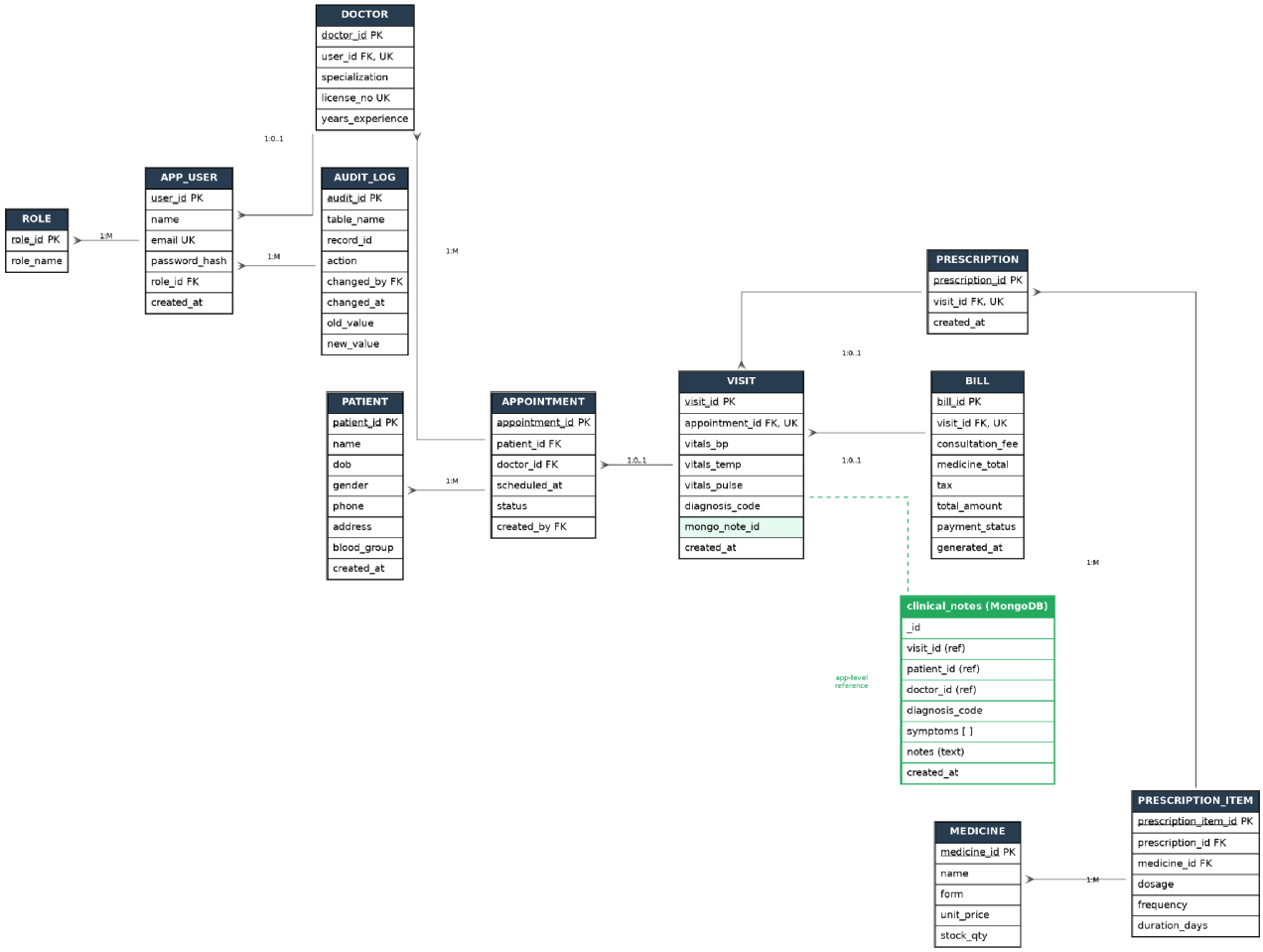


Figure 3.2: ER diagram (relational notation)

4. ER TO RELATIONAL MAPPING

4.1 STEPS OF ALGORITHM FOR CHOSEN PROBLEM

- Step 1 – Regular entities: each strong entity (Role, User, Doctor, Patient, Medicine, Appointment, Visit, Prescription, Bill, Audit Log) becomes a table with its simple attributes; the key attribute becomes the primary key (auto-increment surrogate keys).
- Step 2 – Weak entities: Prescription Item depends on Prescription and Medicine, so it becomes a table whose foreign keys carry the dependency (ON DELETE CASCADE from Prescription).
- Step 3 – 1:1 relationships: Doctor→User, Visit→Appointment, Bill→Visit, Prescription→Visit are mapped by placing a foreign key with a UNIQUE constraint on the dependent side.
- Step 4 – 1:N relationships: Role→User, Patient→Appointment, Doctor→Appointment, Prescription→Prescription Item are mapped by a foreign key on the 'many' side.
- Step 5 – M:N relationships: Prescription ↔ Medicine is resolved by the associative table prescription_item (with dosage, frequency, duration).
- Step 6 – Multivalued/unstructured attributes: free-text notes and symptom lists are not forced into columns; they are stored as a MongoDB document keyed by visit_id and referenced by visit.mongo_note_id.
- Step 7 – Constraints: CHECK (gender, status, price ≥ 0), UNIQUE (email, license number, one bill per visit), FOREIGN KEY … RESTRICT to prevent orphans; indexes on foreign keys and search columns.

Table 4.1: Relational schema (11 tables)

Table	Attributes (* = primary key)	Foreign keys
role	*role_id, role_name	–
app_user	*user_id, name, email, password_hash, role_id, created_at	role_id → role
doctor	*doctor_id, user_id, specialization, license_no, experience_years	user_id → app_user
patient	*patient_id, name, gender, dob, phone, address, blood_group	–
appointment	*appointment_id, patient_id, doctor_id, scheduled_at, status, created_by	patient, doctor, app_user
visit	*visit_id, appointment_id, vitals_bp, vitals_temp, vitals_pulse, diagnosis_code, mongo_note_id	appointment_id → appointment
medicine	*medicine_id, name, form, unit_price, stock_qty	–
prescription	*prescription_id, visit_id, created_at	visit_id → visit
prescription_item	*item_id, prescription_id, medicine_id, dosage, frequency, duration_days	prescription, medicine
bill	*bill_id, visit_id, consultation_fee, medicine_total, tax, total_amount, payment_status, generated_at	visit_id → visit
audit_log	*audit_id, table_name, record_id, action, old_values, new_values, changed_at	–

4.2 COMPLETE DIAGRAM OF RELATIONAL MAPPING

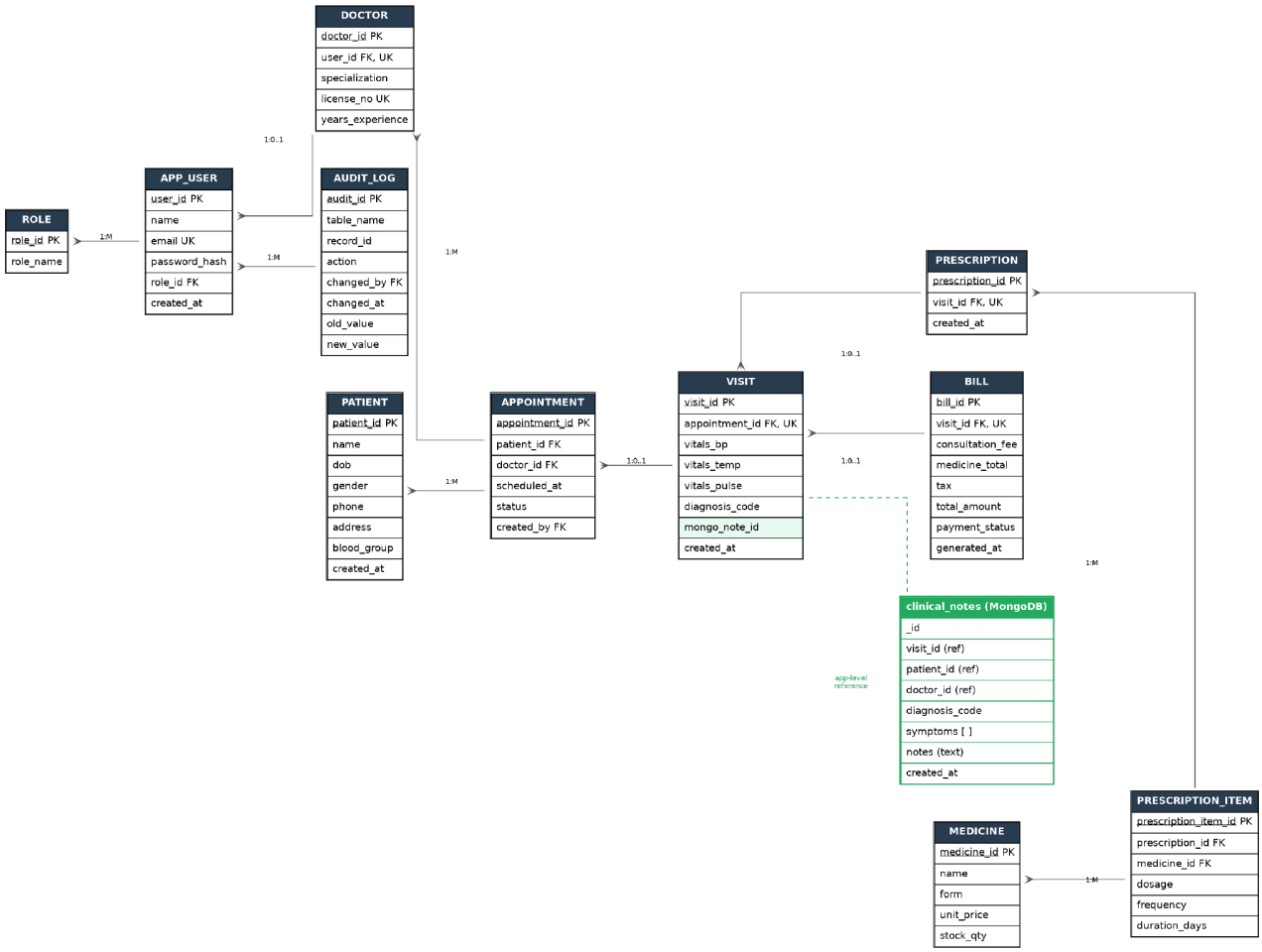


Figure 4.1: Relational mapping of all 11 tables with keys

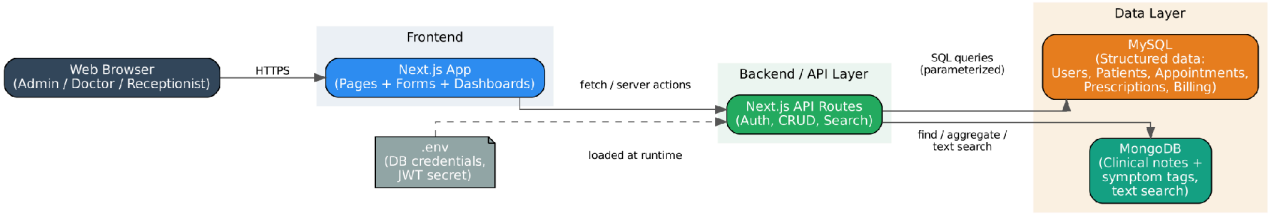


Figure 4.2: System architecture: browser, Next.js API, MySQL and MongoDB

5. DDL STATEMENTS

The schema is created by sql/ddl_schema_mysql.sql. It contains eleven CREATE TABLE statements with primary keys, foreign keys, CHECK and UNIQUE constraints and indexes. The complete script is listed below, followed by screenshots of its execution and of the resulting structure in MySQL 8.0.

```
CREATE DATABASE IF NOT EXISTS medirecord;
USE medirecord;

CREATE TABLE role (
    role_id     INT AUTO_INCREMENT PRIMARY KEY,
    role_name   VARCHAR(30) NOT NULL UNIQUE,
    CHECK (role_name IN ('Admin', 'Doctor', 'Receptionist'))
)

CREATE TABLE app_user (
    user_id         INT AUTO_INCREMENT PRIMARY KEY,
    name            VARCHAR(100) NOT NULL,
    email           VARCHAR(150) NOT NULL UNIQUE,
    password_hash   VARCHAR(255) NOT NULL,
    role_id         INT NOT NULL,
    created_at      TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (role_id) REFERENCES role(role_id) ON DELETE RESTRICT
)

CREATE INDEX idx_user_email ON app_user(email);

CREATE TABLE doctor (
    doctor_id           INT AUTO_INCREMENT PRIMARY KEY,
    user_id             INT NOT NULL UNIQUE,
    specialization      VARCHAR(100) NOT NULL,
    license_no          VARCHAR(50) NOT NULL UNIQUE,
    years_experience    INT NOT NULL DEFAULT 0,
    FOREIGN KEY (user_id) REFERENCES app_user(user_id) ON DELETE CASCADE,
    CHECK (years_experience >= 0)
)

CREATE TABLE patient (
    patient_id      INT AUTO_INCREMENT PRIMARY KEY,
    name            VARCHAR(100) NOT NULL,
    dob             DATE NOT NULL,
    gender          VARCHAR(10) NOT NULL,
    phone           VARCHAR(15) NOT NULL,
    address         VARCHAR(255),
    blood_group     VARCHAR(5),
    created_at      TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CHECK (gender IN ('Male', 'Female', 'Other'))
)

CREATE INDEX idx_patient_phone ON patient(phone);

CREATE TABLE appointment (
    appointment_id  INT AUTO_INCREMENT PRIMARY KEY,
    patient_id      INT NOT NULL,
    doctor_id       INT NOT NULL,
    scheduled_at    TIMESTAMP NOT NULL,
    status          VARCHAR(20) NOT NULL DEFAULT 'scheduled',
    created_by      INT NOT NULL,
    FOREIGN KEY (patient_id) REFERENCES patient(patient_id) ON DELETE RESTRICT,
    FOREIGN KEY (doctor_id) REFERENCES doctor(doctor_id) ON DELETE RESTRICT,
    FOREIGN KEY (created_by) REFERENCES app_user(user_id) ON DELETE RESTRICT,
    CHECK (status IN ('scheduled', 'completed', 'cancelled'))
)

CREATE INDEX idx_appointment_scheduled_at ON appointment(scheduled_at);
CREATE INDEX idx_appointment_doctor ON appointment(doctor_id);
CREATE INDEX idx_appointment_patient ON appointment(patient_id);

CREATE TABLE visit (
```

```
    visit_id            INT AUTO_INCREMENT PRIMARY KEY,
    appointment_id      INT NOT NULL UNIQUE,
    vitals_bp           VARCHAR(15),
    vitals_temp         DECIMAL(4,1),
    vitals_pulse        INT,
    diagnosis_code      VARCHAR(20),
    mongo_note_id       VARCHAR(50),   -- references clinical_notes._id in MongoDB
    created_at          TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (appointment_id) REFERENCES appointment(appointment_id) ON DELETE RESTRICT
)

CREATE INDEX idx_visit_diagnosis_code ON visit(diagnosis_code);
CREATE INDEX idx_visit_mongo_note_id ON visit(mongo_note_id);

CREATE TABLE medicine (
    medicine_id     INT AUTO_INCREMENT PRIMARY KEY,
    name            VARCHAR(150) NOT NULL,
    form            VARCHAR(20) NOT NULL,
    unit_price      DECIMAL(10,2) NOT NULL,
    stock_qty       INT NOT NULL DEFAULT 0,
    CHECK (form IN ('tablet', 'syrup', 'injection', 'ointment', 'other')),
    CHECK (unit_price >= 0),
    CHECK (stock_qty >= 0)
)

CREATE TABLE prescription (
    prescription_id     INT AUTO_INCREMENT PRIMARY KEY,
    visit_id            INT NOT NULL UNIQUE,
    created_at          TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (visit_id) REFERENCES visit(visit_id) ON DELETE RESTRICT
)

CREATE TABLE prescription_item (
    prescription_item_id   INT AUTO_INCREMENT PRIMARY KEY,
    prescription_id        INT NOT NULL,
    medicine_id             INT NOT NULL,
    dosage                  VARCHAR(50) NOT NULL,
    frequency               VARCHAR(50) NOT NULL,
    duration_days           INT NOT NULL,
    FOREIGN KEY (prescription_id) REFERENCES prescription(prescription_id) ON DELETE CASCADE,
    FOREIGN KEY (medicine_id) REFERENCES medicine(medicine_id) ON DELETE RESTRICT,
    CHECK (duration_days > 0)
)

CREATE INDEX idx_prescription_item_prescription ON prescription_item(prescription_id);

CREATE TABLE bill (
    bill_id             INT AUTO_INCREMENT PRIMARY KEY,
    visit_id            INT NOT NULL UNIQUE,
    consultation_fee    DECIMAL(10,2) NOT NULL DEFAULT 0,
    medicine_total      DECIMAL(10,2) NOT NULL DEFAULT 0,
    tax                 DECIMAL(10,2) NOT NULL DEFAULT 0,
    total_amount        DECIMAL(10,2) NOT NULL,
    payment_status      VARCHAR(20) NOT NULL DEFAULT 'pending',
    generated_at        TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (visit_id) REFERENCES visit(visit_id) ON DELETE RESTRICT,
    CHECK (consultation_fee >= 0),
    CHECK (medicine_total >= 0),
    CHECK (tax >= 0),
    CHECK (total_amount >= 0),
    CHECK (payment_status IN ('pending', 'paid', 'refunded'))
)

CREATE TABLE audit_log (
    audit_id        INT AUTO_INCREMENT PRIMARY KEY,
    table_name      VARCHAR(50) NOT NULL,
    record_id       INT NOT NULL,
    action          VARCHAR(10) NOT NULL,
    changed_by      INT,
    changed_at      TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    old_value       JSON,
    new_value       JSON,
    FOREIGN KEY (changed_by) REFERENCES app_user(user_id) ON DELETE SET NULL,
```

```
    CHECK (action IN ('INSERT', 'UPDATE', 'DELETE'))
)

CREATE INDEX idx_audit_table_record ON audit_log(table_name, record_id);

INSERT INTO role (role_name) VALUES ('Admin'), ('Doctor'), ('Receptionist');
```

```
mysql> DESCRIBE app_user;
+---------------+--------------+------+-----+-------------------+-------------------+
| Field         | Type         | Null | Key | Default           | Extra             |
+---------------+--------------+------+-----+-------------------+-------------------+
| user_id       | int          | NO   | PRI | NULL              | auto_increment    |
| name          | varchar(100) | NO   |     | NULL              |                   |
| email         | varchar(150) | NO   | UNI | NULL              |                   |
| password_hash | varchar(255) | NO   |     | NULL              |                   |
| role_id       | int          | NO   | MUL | NULL              |                   |
| created_at    | timestamp    | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+---------------+--------------+------+-----+-------------------+-------------------+
6 rows in set (0.00 sec)
```

```
mysql> CREATE TABLE doctor (
    ->     doctor_id           INT AUTO_INCREMENT PRIMARY KEY,
    ->     user_id             INT NOT NULL UNIQUE,
    ->     specialization      VARCHAR(100) NOT NULL,
    ->     license_no          VARCHAR(50) NOT NULL UNIQUE,
    ->     years_experience    INT NOT NULL DEFAULT 0,
    ->     FOREIGN KEY (user_id) REFERENCES app_user(user_id) ON DELETE CASCADE,
    ->     CHECK (years_experience >= 0)
    -> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.09 sec)

mysql> CREATE TABLE patient (
    ->     patient_id      INT AUTO_INCREMENT PRIMARY KEY,
    ->     name            VARCHAR(100) NOT NULL,
    ->     dob             DATE NOT NULL,
    ->     gender          VARCHAR(10) NOT NULL,
    ->     phone           VARCHAR(15) NOT NULL,
    ->     address         VARCHAR(255),
    ->     blood_group     VARCHAR(5),
    ->     created_at      TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ->     CHECK (gender IN ('Male', 'Female', 'Other'))
    -> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> CREATE INDEX idx_patient_phone ON patient(phone);
Query OK, 0 rows affected (0.05 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

Figure 5.1: Creating the database and the first tables

```
mysql> CREATE TABLE appointment (
    ->     appointment_id  INT AUTO_INCREMENT PRIMARY KEY,
    ->     patient_id      INT NOT NULL,
    ->     doctor_id       INT NOT NULL,
    ->     scheduled_at    TIMESTAMP NOT NULL,
    ->     status          VARCHAR(20) NOT NULL DEFAULT 'scheduled',
    ->     created_by      INT NOT NULL,
    ->     FOREIGN KEY (patient_id) REFERENCES patient(patient_id) ON DELETE RESTRICT,
    ->     FOREIGN KEY (doctor_id) REFERENCES doctor(doctor_id) ON DELETE RESTRICT,
    ->     FOREIGN KEY (created_by) REFERENCES app_user(user_id) ON DELETE RESTRICT,
    ->     CHECK (status IN ('scheduled', 'completed', 'cancelled'))
    -> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.10 sec)

mysql> CREATE INDEX idx_appointment_scheduled_at ON appointment(scheduled_at);
Query OK, 0 rows affected (0.03 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> CREATE INDEX idx_appointment_doctor ON appointment(doctor_id);
Query OK, 0 rows affected (0.03 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> CREATE INDEX idx_appointment_patient ON appointment(patient_id);
Query OK, 0 rows affected (0.04 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> CREATE TABLE visit (
    ->     visit_id            INT AUTO_INCREMENT PRIMARY KEY,
    ->     appointment_id      INT NOT NULL UNIQUE,
    ->     vitals_bp           VARCHAR(15),
    ->     vitals_temp         DECIMAL(4,1),
    ->     vitals_pulse        INT,
    ->     diagnosis_code      VARCHAR(20),
    ->     mongo_note_id       VARCHAR(50),   -- references clinical_notes._id in MongoDB
    ->     created_at          TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ->     FOREIGN KEY (appointment_id) REFERENCES appointment(appointment_id) ON DELETE RESTRICT
    -> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.09 sec)
```

Figure 5.2: Tables created without errors

```
mysql> CREATE INDEX idx_visit_diagnosis_code ON visit(diagnosis_code);
Query OK, 0 rows affected (0.03 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> CREATE INDEX idx_visit_mongo_note_id ON visit(mongo_note_id);
Query OK, 0 rows affected (0.06 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> CREATE TABLE medicine (
    ->     medicine_id     INT AUTO_INCREMENT PRIMARY KEY,
    ->     name            VARCHAR(150) NOT NULL,
    ->     form            VARCHAR(20) NOT NULL,
    ->     unit_price      DECIMAL(10,2) NOT NULL,
    ->     stock_qty       INT NOT NULL DEFAULT 0,
    ->     CHECK (form IN ('tablet', 'syrup', 'injection', 'ointment', 'other')),
    ->     CHECK (unit_price >= 0),
    ->     CHECK (stock_qty >= 0)
    -> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.05 sec)

mysql> CREATE TABLE prescription (
    ->     prescription_id     INT AUTO_INCREMENT PRIMARY KEY,
    ->     visit_id            INT NOT NULL UNIQUE,
    ->     created_at          TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ->     FOREIGN KEY (visit_id) REFERENCES visit(visit_id) ON DELETE RESTRICT
    -> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.09 sec)

mysql> CREATE TABLE prescription_item (
    ->     prescription_item_id   INT AUTO_INCREMENT PRIMARY KEY,
    ->     prescription_id        INT NOT NULL,
    ->     medicine_id             INT NOT NULL,
    ->     dosage                  VARCHAR(50) NOT NULL,
    ->     frequency               VARCHAR(50) NOT NULL,
    ->     duration_days           INT NOT NULL,
    ->     FOREIGN KEY (prescription_id) REFERENCES prescription(prescription_id) ON DELETE CASCADE,
    ->     FOREIGN KEY (medicine_id) REFERENCES medicine(medicine_id) ON DELETE RESTRICT,
    ->     CHECK (duration_days > 0)
    -> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.10 sec)
```

Figure 5.3: SHOW TABLES – all 11 tables present

```
mysql> CREATE TABLE bill (
    ->     bill_id             INT AUTO_INCREMENT PRIMARY KEY,
    ->     visit_id            INT NOT NULL UNIQUE,
    ->     consultation_fee    DECIMAL(10,2) NOT NULL DEFAULT 0,
    ->     medicine_total      DECIMAL(10,2) NOT NULL DEFAULT 0,
    ->     tax                 DECIMAL(10,2) NOT NULL DEFAULT 0,
    ->     total_amount        DECIMAL(10,2) NOT NULL,
    ->     payment_status      VARCHAR(20) NOT NULL DEFAULT 'pending',
    ->     generated_at        TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ->     FOREIGN KEY (visit_id) REFERENCES visit(visit_id) ON DELETE RESTRICT,
    ->     CHECK (consultation_fee >= 0),
    ->     CHECK (medicine_total >= 0),
    ->     CHECK (tax >= 0),
    ->     CHECK (total_amount >= 0),
    ->     CHECK (payment_status IN ('pending', 'paid', 'refunded'))
    -> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.07 sec)

mysql> CREATE TABLE audit_log (
    ->     audit_id        INT AUTO_INCREMENT PRIMARY KEY,
    ->     table_name      VARCHAR(50) NOT NULL,
    ->     record_id       INT NOT NULL,
    ->     action          VARCHAR(10) NOT NULL,
    ->     changed_by      INT,
    ->     changed_at      TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ->     old_value       JSON,
    ->     new_value       JSON,
    ->     FOREIGN KEY (changed_by) REFERENCES app_user(user_id) ON DELETE SET NULL,
    ->     CHECK (action IN ('INSERT', 'UPDATE', 'DELETE'))
    -> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.09 sec)
```

```
mysql> SHOW TABLES;
+----------------------+
| Tables_in_medirecord |
+----------------------+
| app_user             |
| appointment          |
| audit_log            |
| bill                 |
| doctor               |
| medicine             |
| patient              |
| prescription         |
| prescription_item    |
| role                 |
| visit                |
+----------------------+
11 rows in set (0.00 sec)
```

Figure 5.4: DESCRIBE of core tables

```
mysql> DESCRIBE doctor;
+------------------+--------------+------+-----+---------+----------------+
| Field            | Type         | Null | Key | Default | Extra          |
+------------------+--------------+------+-----+---------+----------------+
| doctor_id        | int          | NO   | PRI | NULL    | auto_increment |
| user_id          | int          | NO   | UNI | NULL    |                |
| specialization   | varchar(100) | NO   |     | NULL    |                |
| license_no       | varchar(50)  | NO   | UNI | NULL    |                |
| years_experience | int          | NO   |     | 0       |                |
+------------------+--------------+------+-----+---------+----------------+
5 rows in set (0.00 sec)

mysql> DESCRIBE patient;
+-------------+--------------+------+-----+-------------------+-------------------+
| Field       | Type         | Null | Key | Default           | Extra             |
+-------------+--------------+------+-----+-------------------+-------------------+
| patient_id  | int          | NO   | PRI | NULL              | auto_increment    |
| name        | varchar(100) | NO   |     | NULL              |                   |
| dob         | date         | NO   |     | NULL              |                   |
| gender      | varchar(10)  | NO   |     | NULL              |                   |
| phone       | varchar(15)  | NO   | MUL | NULL              |                   |
| address     | varchar(255) | YES  |     | NULL              |                   |
| blood_group | varchar(5)   | YES  |     | NULL              |                   |
| created_at  | timestamp    | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+-------------+--------------+------+-----+-------------------+-------------------+
8 rows in set (0.00 sec)

mysql> DESCRIBE appointment;
+----------------+-------------+------+-----+-----------+----------------+
| Field          | Type        | Null | Key | Default   | Extra          |
+----------------+-------------+------+-----+-----------+----------------+
| appointment_id | int         | NO   | PRI | NULL      | auto_increment |
| patient_id     | int         | NO   | MUL | NULL      |                |
| doctor_id      | int         | NO   | MUL | NULL      |                |
| scheduled_at   | timestamp   | NO   | MUL | NULL      |                |
| status         | varchar(20) | NO   |     | scheduled |                |
| created_by     | int         | NO   | MUL | NULL      |                |
+----------------+-------------+------+-----+-----------+----------------+
6 rows in set (0.00 sec)
```

Figure 5.5: Constraints and foreign keys (SHOW CREATE TABLE)

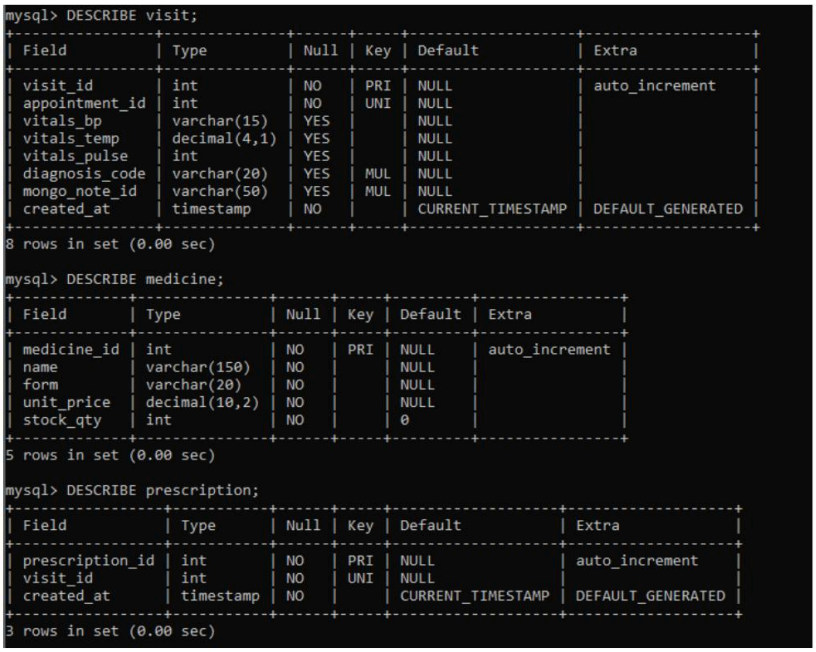


Figure 5.6: Index listing

5.1 Tables and views in the database

```
mysql> prompt pes2ug24cs918 [\d] >
PROMPT set to 'pes2ug24cs918 [\d] >'
pes2ug24cs918 [(none)] >use medirecord;
Database changed
pes2ug24cs918 [medirecord] >show tables;
+------------------------+
| Tables_in_medirecord   |
+------------------------+
| app_user               |
| appointment            |
| audit_log              |
| bill                   |
| doctor                 |
| doctor_dashboard_view  |
| doctor_revenue_summary |
| medicine               |
| patient                |
| prescription           |
| prescription_item      |
| role                   |
| visit                  |
+------------------------+
13 rows in set (0.01 sec)
```

Figure 5.7: SHOW TABLES: 11 tables and 2 views (13 rows)

5.2 Table structures (DESCRIBE)

```
pes2ug24cs918 [medirecord] >desc role;
+-----------+-------------+------+-----+---------+----------------+
| Field     | Type        | Null | Key | Default | Extra          |
+-----------+-------------+------+-----+---------+----------------+
| role_id   | int         | NO   | PRI | NULL    | auto_increment |
| role_name | varchar(30) | NO   | UNI | NULL    |                |
+-----------+-------------+------+-----+---------+----------------+
2 rows in set (0.00 sec)

pes2ug24cs918 [medirecord] >desc app_user;
+---------------+--------------+------+-----+-------------------+-------------------+
| Field         | Type         | Null | Key | Default           | Extra             |
+---------------+--------------+------+-----+-------------------+-------------------+
| user_id       | int          | NO   | PRI | NULL              | auto_increment    |
| name          | varchar(100) | NO   |     | NULL              |                   |
| email         | varchar(150) | NO   | UNI | NULL              |                   |
| password_hash | varchar(255) | NO   |     | NULL              |                   |
| role_id       | int          | NO   | MUL | NULL              |                   |
| created_at    | timestamp    | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+---------------+--------------+------+-----+-------------------+-------------------+
6 rows in set (0.00 sec)

pes2ug24cs918 [medirecord] >desc doctor;
+------------------+--------------+------+-----+---------+----------------+
| Field            | Type         | Null | Key | Default | Extra          |
+------------------+--------------+------+-----+---------+----------------+
| doctor_id        | int          | NO   | PRI | NULL    | auto_increment |
| user_id          | int          | NO   | UNI | NULL    |                |
| specialization   | varchar(100) | NO   |     | NULL    |                |
| license_no       | varchar(50)  | NO   | UNI | NULL    |                |
| years_experience | int          | NO   |     | 0       |                |
+------------------+--------------+------+-----+---------+----------------+
5 rows in set (0.00 sec)
```

Figure 5.8: DESCRIBE role, app_user and doctor

```
pes2ug24cs918 [medirecord] >desc patient;
+-------------+--------------+------+-----+-------------------+-------------------+
| Field       | Type         | Null | Key | Default           | Extra             |
+-------------+--------------+------+-----+-------------------+-------------------+
| patient_id  | int          | NO   | PRI | NULL              | auto_increment    |
| name        | varchar(100) | NO   |     | NULL              |                   |
| dob         | date         | NO   |     | NULL              |                   |
| gender      | varchar(10)  | NO   |     | NULL              |                   |
| phone       | varchar(15)  | NO   | MUL | NULL              |                   |
| address     | varchar(255) | YES  |     | NULL              |                   |
| blood_group | varchar(5)   | YES  |     | NULL              |                   |
| created_at  | timestamp    | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+-------------+--------------+------+-----+-------------------+-------------------+
8 rows in set (0.00 sec)

pes2ug24cs918 [medirecord] >desc appointment;
+----------------+-------------+------+-----+-----------+----------------+
| Field          | Type        | Null | Key | Default   | Extra          |
+----------------+-------------+------+-----+-----------+----------------+
| appointment_id | int         | NO   | PRI | NULL      | auto_increment |
| patient_id     | int         | NO   | MUL | NULL      |                |
| doctor_id      | int         | NO   | MUL | NULL      |                |
| scheduled_at   | timestamp   | NO   | MUL | NULL      |                |
| status         | varchar(20) | NO   |     | scheduled |                |
| created_by     | int         | NO   | MUL | NULL      |                |
+----------------+-------------+------+-----+-----------+----------------+
6 rows in set (0.00 sec)
```

Figure 5.9: DESCRIBE patient

```
pes2ug24cs918 [medirecord] >desc visit;
+----------------+--------------+------+-----+-------------------+-------------------+
| Field          | Type         | Null | Key | Default           | Extra             |
+----------------+--------------+------+-----+-------------------+-------------------+
| visit_id       | int          | NO   | PRI | NULL              | auto_increment    |
| appointment_id | int          | NO   | UNI | NULL              |                   |
| vitals_bp      | varchar(15)  | YES  |     | NULL              |                   |
| vitals_temp    | decimal(4,1) | YES  |     | NULL              |                   |
| vitals_pulse   | int          | YES  |     | NULL              |                   |
| diagnosis_code | varchar(20)  | YES  | MUL | NULL              |                   |
| mongo_note_id  | varchar(50)  | YES  | MUL | NULL              |                   |
| created_at     | timestamp    | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+----------------+--------------+------+-----+-------------------+-------------------+
8 rows in set (0.00 sec)

pes2ug24cs918 [medirecord] >desc medicine;
+-------------+---------------+------+-----+---------+----------------+
| Field       | Type          | Null | Key | Default | Extra          |
+-------------+---------------+------+-----+---------+----------------+
| medicine_id | int           | NO   | PRI | NULL    | auto_increment |
| name        | varchar(150)  | NO   |     | NULL    |                |
| form        | varchar(20)   | NO   |     | NULL    |                |
| unit_price  | decimal(10,2) | NO   |     | NULL    |                |
| stock_qty   | int           | NO   |     | 0       |                |
+-------------+---------------+------+-----+---------+----------------+
5 rows in set (0.00 sec)
```

Figure 5.10: DESCRIBE visit and medicine

```
pes2ug24cs918 [medirecord] >desc prescription;
+-----------------+-----------+------+-----+-------------------+-------------------+
| Field           | Type      | Null | Key | Default           | Extra             |
+-----------------+-----------+------+-----+-------------------+-------------------+
| prescription_id | int       | NO   | PRI | NULL              | auto_increment    |
| visit_id        | int       | NO   | UNI | NULL              |                   |
| created_at      | timestamp | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+-----------------+-----------+------+-----+-------------------+-------------------+
3 rows in set (0.00 sec)

pes2ug24cs918 [medirecord] >desc prescription_item;
+---------------------+-------------+------+-----+---------+----------------+
| Field               | Type        | Null | Key | Default | Extra          |
+---------------------+-------------+------+-----+---------+----------------+
| prescription_item_id | int         | NO   | PRI | NULL    | auto_increment |
| prescription_id      | int         | NO   | MUL | NULL    |                |
| medicine_id          | int         | NO   | MUL | NULL    |                |
| dosage               | varchar(50) | NO   |     | NULL    |                |
| frequency            | varchar(50) | NO   |     | NULL    |                |
| duration_days        | int         | NO   |     | NULL    |                |
+---------------------+-------------+------+-----+---------+----------------+
6 rows in set (0.00 sec)
```

Figure 5.11: DESCRIBE prescription and prescription_item

```
pes2ug24cs918 [medirecord] >desc bill;
+-----------------+---------------+------+-----+-------------------+-------------------+
| Field           | Type          | Null | Key | Default           | Extra             |
+-----------------+---------------+------+-----+-------------------+-------------------+
| bill_id         | int           | NO   | PRI | NULL              | auto_increment    |
| visit_id        | int           | NO   | UNI | NULL              |                   |
| consultation_fee | decimal(10,2) | NO   |     | 0.00              |                   |
| medicine_total  | decimal(10,2) | NO   |     | 0.00              |                   |
| tax             | decimal(10,2) | NO   |     | 0.00              |                   |
| total_amount    | decimal(10,2) | NO   |     | NULL              |                   |
| payment_status  | varchar(20)   | NO   |     | pending           |                   |
| generated_at    | timestamp     | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+-----------------+---------------+------+-----+-------------------+-------------------+
8 rows in set (0.00 sec)

pes2ug24cs918 [medirecord] >desc audit_log;
+------------+-------------+------+-----+-------------------+-------------------+
| Field      | Type        | Null | Key | Default           | Extra             |
+------------+-------------+------+-----+-------------------+-------------------+
| audit_id   | int         | NO   | PRI | NULL              | auto_increment    |
| table_name | varchar(50) | NO   | MUL | NULL              |                   |
| record_id  | int         | NO   |     | NULL              |                   |
| action     | varchar(10) | NO   |     | NULL              |                   |
| changed_by | int         | YES  | MUL | NULL              |                   |
| changed_at | timestamp   | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
| old_value  | json        | YES  |     | NULL              |                   |
| new_value  | json        | YES  |     | NULL              |                   |
+------------+-------------+------+-----+-------------------+-------------------+
8 rows in set (0.00 sec)
```

Figure 5.12: DESCRIBE bill and audit_log

5.3 Constraints and foreign keys

```
pes2ug24cs918 [medirecord] >SHOW CREATE TABLE appointment\G
*************************** 1. row ***************************
       Table: appointment
Create Table: CREATE TABLE `appointment` (
  `appointment_id` int NOT NULL AUTO_INCREMENT,
  `patient_id` int NOT NULL,
  `doctor_id` int NOT NULL,
  `scheduled_at` timestamp NOT NULL,
  `status` varchar(20) NOT NULL DEFAULT 'scheduled',
  `created_by` int NOT NULL,
  PRIMARY KEY (`appointment_id`),
  KEY `created_by` (`created_by`),
  KEY `idx_appointment_scheduled_at` (`scheduled_at`),
  KEY `idx_appointment_doctor` (`doctor_id`),
  KEY `idx_appointment_patient` (`patient_id`),
  CONSTRAINT `appointment_ibfk_1` FOREIGN KEY (`patient_id`) REFERENCES `patient` (`patient_id`) ON DELETE RESTRICT,
  CONSTRAINT `appointment_ibfk_2` FOREIGN KEY (`doctor_id`) REFERENCES `doctor` (`doctor_id`) ON DELETE RESTRICT,
  CONSTRAINT `appointment_ibfk_3` FOREIGN KEY (`created_by`) REFERENCES `app_user` (`user_id`) ON DELETE RESTRICT,
  CONSTRAINT `appointment_chk_1` CHECK ((`status` in (_utf8mb4'scheduled',_utf8mb4'completed',_utf8mb4'cancelled')))
) ENGINE=InnoDB AUTO_INCREMENT=12 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
1 row in set (0.00 sec)
```

Figure 5.13: SHOW CREATE TABLE appointment: primary key, foreign keys (RESTRICT) and CHECK

5.4 ALTER and DROP

```
ALTER TABLE patient ADD COLUMN emergency_contact VARCHAR(15);
ALTER TABLE patient DROP COLUMN emergency_contact;
CREATE TABLE temp_demo (id INT PRIMARY KEY, note VARCHAR(30));
DROP TABLE temp_demo;
```

```
pes2ug24cs918 [medirecord] >ALTER TABLE patient ADD COLUMN emergency_contact VARCHAR(15);
Query OK, 0 rows affected (0.05 sec)
Records: 0  Duplicates: 0  Warnings: 0

pes2ug24cs918 [medirecord] >desc patient;
+-------------------+--------------+------+-----+-------------------+-------------------+
| Field             | Type         | Null | Key | Default           | Extra             |
+-------------------+--------------+------+-----+-------------------+-------------------+
| patient_id        | int          | NO   | PRI | NULL              | auto_increment    |
| name              | varchar(100) | NO   |     | NULL              |                   |
| dob               | date         | NO   |     | NULL              |                   |
| gender            | varchar(10)  | NO   |     | NULL              |                   |
| phone             | varchar(15)  | NO   | MUL | NULL              |                   |
| address           | varchar(255) | YES  |     | NULL              |                   |
| blood_group       | varchar(5)   | YES  |     | NULL              |                   |
| created_at        | timestamp    | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
| emergency_contact | varchar(15)  | YES  |     | NULL              |                   |
+-------------------+--------------+------+-----+-------------------+-------------------+
9 rows in set (0.00 sec)

pes2ug24cs918 [medirecord] >ALTER TABLE patient DROP COLUMN emergency_contact;
Query OK, 0 rows affected (0.04 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

Figure 5.14: ALTER TABLE: add and drop a column

```
pes2ug24cs918 [medirecord] >CREATE TABLE temp_demo (id INT PRIMARY KEY, note VARCHAR(30));
Query OK, 0 rows affected (0.05 sec)

pes2ug24cs918 [medirecord] >show tables;
+------------------------+
| Tables_in_medirecord   |
+------------------------+
| app_user               |
| appointment            |
| audit_log              |
| bill                   |
| doctor                 |
| doctor_dashboard_view  |
| doctor_revenue_summary |
| medicine               |
| patient                |
| prescription           |
| prescription_item      |
| role                   |
| temp_demo              |
| visit                  |
+------------------------+
14 rows in set (0.00 sec)
```

Figure 5.15: CREATE TABLE temp_demo (13 → 14 objects)

```
pes2ug24cs918 [medirecord] >DROP TABLE temp_demo;
Query OK, 0 rows affected (0.02 sec)

pes2ug24cs918 [medirecord] >show tables;
+------------------------+
| Tables_in_medirecord   |
+------------------------+
| app_user               |
| appointment            |
| audit_log              |
| bill                   |
| doctor                 |
| doctor_dashboard_view  |
| doctor_revenue_summary |
| medicine               |
| patient                |
| prescription           |
| prescription_item      |
| role                   |
| visit                  |
+------------------------+
13 rows in set (0.00 sec)
```

Figure 5.16: DROP TABLE temp_demo (back to 13 objects)

6. DML STATEMENTS

Demo data is loaded by scripts/seed-demo.js through the same code paths the application uses (bcrypt hashing, transactions, the generate_bill procedure and triggers). Representative INSERT statements and the resulting table contents follow.

```
C:\Users\Rakshitha\Desktop\MediRecord\web>node scripts\seed-demo.js
◇ injected env (8) from .env.local

Demo data loaded:
  users           4
  doctors         2
  patients        8
  medicines       12
  appointments    11
  visits          8
  prescriptions   8
  bills           8
  bills_paid      5
  audit_log_rows  9

Logins:
  Admin         admin@medirecord.com          rkb@123
  Receptionist  reception@medirecord.com      password123
  Doctor        priya.sharma@medirecord.com   doctor@123   (General Medicine)
  Doctor        arvind.rao@medirecord.com     doctor@123   (Pulmonology)
```

Figure 6.1: Loading the demo data (seed-demo.js)

```
START TRANSACTION;
INSERT INTO patient (name, dob, gender, phone, address, blood_group)
VALUES ('Demo Patient', '1995-05-05', 'Female', '9000000001', 'Bengaluru', 'A+');
INSERT INTO appointment (patient_id, doctor_id, scheduled_at, status, created_by)
VALUES (LAST_INSERT_ID(), 1, NOW(), 'scheduled', 2);
ROLLBACK;
```

```
pes2ug24cs918 [medirecord] >START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

pes2ug24cs918 [medirecord] >INSERT INTO patient (name, dob, gender, phone, address, blood_group)
    -> VALUES ('Demo Patient', '1995-05-05', 'Female', '9000000001', 'Bengaluru', 'A+');
Query OK, 1 row affected (0.00 sec)

pes2ug24cs918 [medirecord] >INSERT INTO appointment (patient_id, doctor_id, scheduled_at, status, created_by)
    -> VALUES (LAST_INSERT_ID(), 1, NOW(), 'scheduled', 2);
Query OK, 1 row affected (0.00 sec)

pes2ug24cs918 [medirecord] >SELECT * FROM patient ORDER BY patient_id DESC LIMIT 1;
+------------+-------------+------------+--------+------------+-----------+------------+---------------------+
| patient_id | name        | dob        | gender | phone      | address   | blood_group | created_at          |
+------------+-------------+------------+--------+------------+-----------+------------+---------------------+
|          9 | Demo Patient | 1995-05-05 | Female | 9000000001 | Bengaluru | A+          | 2026-11-02 22:43:41 |
+------------+-------------+------------+--------+------------+-----------+------------+---------------------+
1 row in set (0.00 sec)

pes2ug24cs918 [medirecord] >SELECT * FROM appointment ORDER BY appointment_id DESC LIMIT 1;
+----------------+------------+-----------+---------------------+-----------+------------+
| appointment_id | patient_id | doctor_id | scheduled_at        | status    | created_by |
+----------------+------------+-----------+---------------------+-----------+------------+
|             12 |          9 |         1 | 2026-11-02 22:43:41 | scheduled |          2 |
+----------------+------------+-----------+---------------------+-----------+------------+
1 row in set (0.00 sec)

pes2ug24cs918 [medirecord] >ROLLBACK;
Query OK, 0 rows affected (0.01 sec)
```

Figure 6.2: INSERT into patient and appointment inside a transaction

```
pes2ug24cs918 [medirecord] >SELECT * FROM role;
+---------+--------------+
| role_id | role_name    |
+---------+--------------+
|       1 | Admin        |
|       2 | Doctor       |
|       3 | Receptionist |
+---------+--------------+
3 rows in set (0.00 sec)

pes2ug24cs918 [medirecord] >SELECT user_id, name, email, role_id FROM app_user;
+---------+----------------+-----------------------------+---------+
| user_id | name           | email                       | role_id |
+---------+----------------+-----------------------------+---------+
|       1 | Admin          | admin@medirecord.com        |       1 |
|       2 | Reception Desk | reception@medirecord.com    |       3 |
|       3 | Priya Sharma   | priya.sharma@medirecord.com |       2 |
|       4 | Arvind Rao     | arvind.rao@medirecord.com   |       2 |
+---------+----------------+-----------------------------+---------+
4 rows in set (0.00 sec)
```

Figure 6.3: role and app_user tables

```
pes2ug24cs918 [medirecord] >SELECT * FROM doctor;
+-----------+---------+------------------+------------+------------------+
| doctor_id | user_id | specialization   | license_no | years_experience |
+-----------+---------+------------------+------------+------------------+
|         1 |       3 | General Medicine | KA98765    |                8 |
|         2 |       4 | Pulmonology      | KA55120    |               14 |
+-----------+---------+------------------+------------+------------------+
2 rows in set (0.00 sec)
```

Figure 6.4: doctor table

```
pes2ug24cs918 [medirecord] >SELECT patient_id, name, gender, dob, phone, blood_group FROM patient;
+------------+---------------+--------+------------+------------+-------------+
| patient_id | name          | gender | dob        | phone      | blood_group |
+------------+---------------+--------+------------+------------+-------------+
|          1 | Ravi Kumar    | Male   | 1990-04-12 | 9876543210 | O+          |
|          2 | Sneha Rao     | Female | 1985-09-23 | 9845012345 | B+          |
|          3 | Karthik Nair  | Male   | 1978-01-30 | 9741098765 | A+          |
|          4 | Anita Desai   | Female | 1996-06-18 | 9822011223 | AB+         |
|          5 | Mohammed Faiz | Male   | 1982-11-02 | 9900112233 | O-          |
|          6 | Lakshmi Iyer  | Female | 1965-03-27 | 9444055667 | B-          |
|          7 | Rohan Gupta   | Male   | 2001-08-09 | 9810099887 | A+          |
|          8 | Divya Menon   | Female | 1993-12-15 | 9605566778 | O+          |
+------------+---------------+--------+------------+------------+-------------+
8 rows in set (0.00 sec)
```

Figure 6.5: patient table

```
pes2ug24cs918 [medirecord] >SELECT * FROM medicine;
+-------------+------------------------+--------+------------+-----------+
| medicine_id | name                   | form   | unit_price | stock_qty |
+-------------+------------------------+--------+------------+-----------+
|           1 | Paracetamol 500mg      | tablet |       2.00 |       500 |
|           2 | Amoxicillin 500mg      | tablet |      12.00 |       300 |
|           3 | Cetirizine 10mg        | tablet |       3.50 |       400 |
|           4 | Salbutamol Inhaler     | other  |      95.00 |       120 |
|           5 | Cough Syrup (Benadryl) | syrup  |      85.00 |       150 |
|           6 | Pantoprazole 40mg      | tablet |       8.00 |       250 |
|           7 | Ibuprofen 400mg        | tablet |       4.00 |       350 |
|           8 | Azithromycin 500mg     | tablet |      30.00 |       200 |
|           9 | Amlodipine 5mg         | tablet |       6.00 |       300 |
|          10 | Sumatriptan 50mg       | tablet |      25.00 |       100 |
|          11 | Montelukast 10mg       | tablet |      14.00 |       200 |
|          12 | Domperidone 10mg       | tablet |       3.00 |       250 |
+-------------+------------------------+--------+------------+-----------+
12 rows in set (0.02 sec)
```

Figure 6.6: medicine table

```
pes2ug24cs918 [medirecord] >SELECT * FROM appointment;
+----------------+------------+-----------+---------------------+-----------+------------+
| appointment_id | patient_id | doctor_id | scheduled_at        | status    | created_by |
+----------------+------------+-----------+---------------------+-----------+------------+
|              1 |          1 |         2 | 2026-10-24 22:32:01 | completed |          2 |
|              2 |          2 |         2 | 2026-10-25 22:32:01 | completed |          2 |
|              3 |          3 |         1 | 2026-10-26 22:32:01 | completed |          2 |
|              4 |          4 |         1 | 2026-10-27 22:32:01 | completed |          2 |
|              5 |          5 |         2 | 2026-10-28 22:32:01 | completed |          2 |
|              6 |          6 |         1 | 2026-10-29 22:32:01 | completed |          2 |
|              7 |          7 |         1 | 2026-10-30 22:32:01 | completed |          2 |
|              8 |          8 |         1 | 2026-10-31 22:32:01 | completed |          2 |
|              9 |          1 |         2 | 2026-11-03 18:32:01 | scheduled |          2 |
|             10 |          3 |         1 | 2026-11-03 22:32:01 | scheduled |          2 |
|             11 |          4 |         1 | 2026-11-04 18:32:01 | scheduled |          2 |
+----------------+------------+-----------+---------------------+-----------+------------+
11 rows in set (0.00 sec)
```

Figure 6.7: appointment table

```
pes2ug24cs918 [medirecord] >SELECT visit_id, appointment_id, vitals_bp, vitals_temp, vitals_pulse, diagnosis_code FROM visit;
+----------+----------------+-----------+-------------+--------------+----------------+
| visit_id | appointment_id | vitals_bp | vitals_temp | vitals_pulse | diagnosis_code |
+----------+----------------+-----------+-------------+--------------+----------------+
|        1 |              1 | 128/82    |        98.4 |           88 | J45            |
|        2 |              2 | 118/76    |        98.6 |           82 | J45            |
|        3 |              3 | 122/80    |       101.2 |           96 | J06            |
|        4 |              4 | 110/70    |        98.2 |           78 | K29            |
|        5 |              5 | 136/88    |        98.8 |           92 | J44            |
|        6 |              6 | 160/100   |        98.4 |           84 | I10            |
|        7 |              7 | 120/78    |       100.4 |           90 | J06            |
|        8 |              8 | 112/72    |        98.0 |           74 | G43            |
+----------+----------------+-----------+-------------+--------------+----------------+
8 rows in set (0.02 sec)
```

Figure 6.8: visit table

```
pes2ug24cs918 [medirecord] >SELECT * FROM prescription;
+-----------------+----------+---------------------+
| prescription_id | visit_id | created_at          |
+-----------------+----------+---------------------+
|               1 |        1 | 2026-11-02 22:32:01 |
|               2 |        2 | 2026-11-02 22:32:01 |
|               3 |        3 | 2026-11-02 22:32:01 |
|               4 |        4 | 2026-11-02 22:32:01 |
|               5 |        5 | 2026-11-02 22:32:01 |
|               6 |        6 | 2026-11-02 22:32:01 |
|               7 |        7 | 2026-11-02 22:32:01 |
|               8 |        8 | 2026-11-02 22:32:01 |
+-----------------+----------+---------------------+
8 rows in set (0.00 sec)
```

Figure 6.9: prescription table

```
pes2ug24cs918 [medirecord] >SELECT * FROM prescription_item;
+----------------------+-----------------+-------------+----------+---------------------+---------------+
| prescription_item_id | prescription_id | medicine_id | dosage   | frequency           | duration_days |
+----------------------+-----------------+-------------+----------+---------------------+---------------+
|                    1 |               1 |           4 | 2 puffs  | 4 times a day       |            14 |
|                    2 |               1 |          11 | 1 tablet | once at night       |            14 |
|                    3 |               2 |           4 | 2 puffs  | as needed           |            10 |
|                    4 |               2 |           3 | 1 tablet | once a day          |             7 |
|                    5 |               3 |           1 | 1 tablet | 3 times a day       |             3 |
|                    6 |               4 |           6 | 1 tablet | before breakfast    |            14 |
|                    7 |               4 |          12 | 1 tablet | twice a day         |             5 |
|                    8 |               5 |           4 | 2 puffs  | 3 times a day       |            30 |
|                    9 |               5 |           8 | 1 tablet | once a day          |             5 |
|                   10 |               6 |           9 | 1 tablet | once a day          |            30 |
|                   11 |               7 |           1 | 1 tablet | 3 times a day       |             3 |
|                   12 |               7 |           5 | 10 ml    | 3 times a day       |             5 |
|                   13 |               8 |          10 | 1 tablet | at onset of headache |             5 |
|                   14 |               8 |          12 | 1 tablet | as needed           |             5 |
+----------------------+-----------------+-------------+----------+---------------------+---------------+
14 rows in set (0.00 sec)
```

Figure 6.10: prescription_item table

```
pes2ug24cs918 [medirecord] >SELECT * FROM bill;
+---------+----------+-----------------+---------------+--------+--------------+----------------+---------------------+
| bill_id | visit_id | consultation_fee | medicine_total | tax    | total_amount | payment_status | generated_at        |
+---------+----------+-----------------+---------------+--------+--------------+----------------+---------------------+
|       1 |        1 |          500.00 |       1526.00 | 101.30 |      2127.30 | paid           | 2026-11-02 22:32:01 |
|       2 |        2 |          500.00 |        974.50 |  73.73 |      1548.23 | paid           | 2026-11-02 22:32:01 |
|       3 |        3 |          300.00 |          6.00 |  15.30 |       321.30 | paid           | 2026-11-02 22:32:01 |
|       4 |        4 |          300.00 |        127.00 |  21.35 |       448.35 | pending        | 2026-11-02 22:32:01 |
|       5 |        5 |          500.00 |       3000.00 | 175.00 |      3675.00 | pending        | 2026-11-02 22:32:01 |
|       6 |        6 |          300.00 |        180.00 |  24.00 |       504.00 | paid           | 2026-11-02 22:32:01 |
|       7 |        7 |          300.00 |        431.00 |  36.55 |       767.55 | pending        | 2026-11-02 22:32:01 |
|       8 |        8 |          300.00 |        140.00 |  22.00 |       462.00 | paid           | 2026-11-02 22:32:01 |
+---------+----------+-----------------+---------------+--------+--------------+----------------+---------------------+
8 rows in set (0.00 sec)
```

Figure 6.11: bill table (generated by stored procedure)

```
pes2ug24cs918 [medirecord] >SELECT 'patient' AS tbl, COUNT(*) AS n FROM patient
    -> UNION SELECT 'doctor', COUNT(*) FROM doctor
    -> UNION SELECT 'appointment', COUNT(*) FROM appointment
    -> UNION SELECT 'visit', COUNT(*) FROM visit
    -> UNION SELECT 'prescription_item', COUNT(*) FROM prescription_item
    -> UNION SELECT 'bill', COUNT(*) FROM bill;
+------------------+----+
| tbl              | n  |
+------------------+----+
| patient          |  8 |
| doctor           |  2 |
| appointment      | 11 |
| visit            |  8 |
| prescription_item | 14 |
| bill             |  8 |
+------------------+----+
6 rows in set (0.00 sec)
```

Figure 6.12: Row count per table

7. QUERIES

All queries below were executed on the demo database. Statements that modify data are wrapped in a transaction and rolled back so that the demo data stays intact.

7.0 JOIN

```
SELECT p.name AS patient, u.name AS doctor, v.diagnosis_code, b.total_amount
FROM visit v JOIN appointment a ON a.appointment_id=v.appointment_id
JOIN patient p ON p.patient_id=a.patient_id
JOIN doctor d ON d.doctor_id=a.doctor_id
JOIN app_user u ON u.user_id=d.user_id
LEFT JOIN bill b ON b.visit_id=v.visit_id;
```

```
pes2ug24cs918 [medirecord] >SELECT p.name AS patient, u.name AS doctor, v.diagnosis_code, b.total_amount
    -> FROM visit v
    -> JOIN appointment a ON a.appointment_id = v.appointment_id
    -> JOIN patient p ON p.patient_id = a.patient_id
    -> JOIN doctor d ON d.doctor_id = a.doctor_id
    -> JOIN app_user u ON u.user_id = d.user_id
    -> LEFT JOIN bill b ON b.visit_id = v.visit_id;
+--------------+--------------+----------------+--------------+
| patient      | doctor       | diagnosis_code | total_amount |
+--------------+--------------+----------------+--------------+
| Ravi Kumar   | Arvind Rao   | J45            |      2127.30 |
| Sneha Rao    | Arvind Rao   | J45            |      1548.23 |
| Karthik Nair | Priya Sharma | J06            |       321.30 |
| Anita Desai  | Priya Sharma | K29            |       448.35 |
| Mohammed Faiz | Arvind Rao  | J44            |      3675.00 |
| Lakshmi Iyer | Priya Sharma | I10            |       504.00 |
| Rohan Gupta  | Priya Sharma | J06            |       767.55 |
| Divya Menon  | Priya Sharma | G43            |       462.00 |
+--------------+--------------+----------------+--------------+
8 rows in set (0.00 sec)
```

Figure 7.1: Five-table JOIN with LEFT JOIN

7.1 SIMPLE QUERY WITH GROUP BY, AGGREGATE

```
SELECT u.name AS doctor, COUNT(*) AS visits
FROM visit v JOIN appointment a ON a.appointment_id=v.appointment_id
JOIN doctor d ON d.doctor_id=a.doctor_id JOIN app_user u ON u.user_id=d.user_id
GROUP BY d.doctor_id HAVING COUNT(*)>=4;
```

```
pes2ug24cs918 [medirecord] >SELECT u.name AS doctor, COUNT(*) AS visits
    -> FROM visit v
    -> JOIN appointment a ON a.appointment_id = v.appointment_id
    -> JOIN doctor d ON d.doctor_id = a.doctor_id
    -> JOIN app_user u ON u.user_id = d.user_id
    -> GROUP BY d.doctor_id
    -> HAVING COUNT(*) >= 4;
+--------------+--------+
| doctor       | visits |
+--------------+--------+
| Priya Sharma |      5 |
+--------------+--------+
1 row in set (0.00 sec)
```

Figure 7.2: GROUP BY with HAVING

ORDER BY with an aggregate

```
SELECT payment_status, COUNT(*) AS bills, SUM(total_amount) AS amount
FROM bill GROUP BY payment_status ORDER BY amount DESC;
```

```
pes2ug24cs918 [medirecord] >SELECT payment_status, COUNT(*) AS bills, SUM(total_amount) AS amount
    -> FROM bill
    -> GROUP BY payment_status
    -> ORDER BY amount DESC;
+----------------+-------+---------+
| payment_status | bills | amount  |
+----------------+-------+---------+
| paid           |     5 | 4962.83 |
| pending        |     3 | 4890.90 |
+----------------+-------+---------+
2 rows in set (0.00 sec)
```

Figure 7.3: Aggregate with ORDER BY

7.2 UPDATE OPERATION

```
START TRANSACTION;
UPDATE bill SET payment_status='paid' WHERE bill_id=4;
SELECT bill_id, payment_status FROM bill WHERE bill_id=4;
ROLLBACK;
```

```
pes2ug24cs918 [medirecord] >START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

pes2ug24cs918 [medirecord] >UPDATE bill SET payment_status = 'paid' WHERE bill_id = 4;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

pes2ug24cs918 [medirecord] >SELECT bill_id, payment_status FROM bill WHERE bill_id = 4;
+---------+----------------+
| bill_id | payment_status |
+---------+----------------+
|       4 | paid           |
+---------+----------------+
1 row in set (0.00 sec)

pes2ug24cs918 [medirecord] >ROLLBACK;
Query OK, 0 rows affected (0.00 sec)
```

Figure 7.4: UPDATE inside a transaction

7.3 DELETE OPERATION

```
START TRANSACTION;
DELETE FROM medicine WHERE medicine_id NOT IN (SELECT medicine_id FROM prescription_item);
SELECT COUNT(*) AS medicines_left FROM medicine;
ROLLBACK;
```

```
pes2ug24cs918 [medirecord] >START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

pes2ug24cs918 [medirecord] >DELETE FROM medicine WHERE medicine_id NOT IN (SELECT medicine_id FROM prescription_item);
Query OK, 2 rows affected (0.00 sec)

pes2ug24cs918 [medirecord] >SELECT COUNT(*) AS medicines_left FROM medicine;
+----------------+
| medicines_left |
+----------------+
|             10 |
+----------------+
1 row in set (0.00 sec)

pes2ug24cs918 [medirecord] >ROLLBACK;
Query OK, 0 rows affected (0.01 sec)

pes2ug24cs918 [medirecord] >SELECT COUNT(*) AS medicines_after_rollback FROM medicine;
+--------------------------+
| medicines_after_rollback |
+--------------------------+
|                       12 |
+--------------------------+
1 row in set (0.00 sec)
```

Figure 7.5: DELETE of never-prescribed medicines (rolled back: 12 medicines again)

7.4 CORRELATED QUERY

```
SELECT b.bill_id, b.total_amount FROM bill b
WHERE b.total_amount > (SELECT AVG(b2.total_amount) FROM bill b2 WHERE b2.visit_id <> b.visit_id);
```

```
pes2ug24cs918 [medirecord] >SELECT b.bill_id, b.total_amount
    -> FROM bill b
    -> WHERE b.total_amount > (SELECT AVG(b2.total_amount) FROM bill b2 WHERE b2.visit_id <> b.visit_id);
+---------+--------------+
| bill_id | total_amount |
+---------+--------------+
|       1 |      2127.30 |
|       2 |      1548.23 |
|       5 |      3675.00 |
+---------+--------------+
3 rows in set (0.00 sec)
```

Figure 7.6: Correlated subquery: bills above the average of the others

7.5 NESTED QUERY

```
SELECT name FROM patient WHERE patient_id IN
 (SELECT patient_id FROM appointment WHERE status='scheduled');
```

```
pes2ug24cs918 [medirecord] >SELECT name FROM patient
    -> WHERE patient_id IN (SELECT patient_id FROM appointment WHERE status = 'scheduled');
+--------------+
| name         |
+--------------+
| Ravi Kumar   |
| Karthik Nair |
| Anita Desai  |
+--------------+
3 rows in set (0.00 sec)
```

Figure 7.7: Nested query with IN

LEFT JOIN: patients with no appointment

```
SELECT p.name FROM patient p
LEFT JOIN appointment a ON a.patient_id=p.patient_id
WHERE a.appointment_id IS NULL;
```

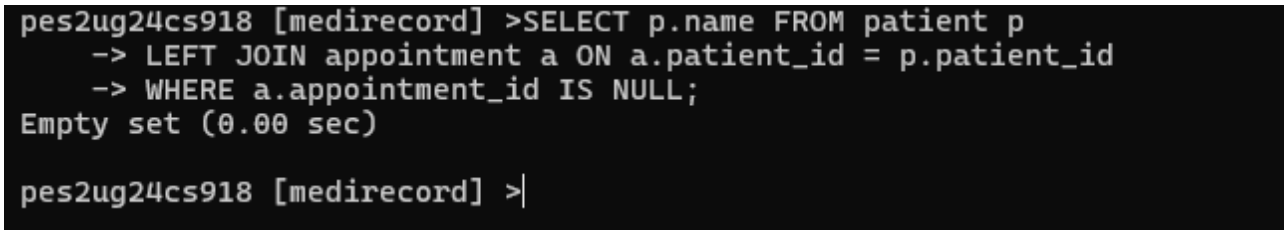


Figure 7.8: LEFT JOIN returns an empty set: every demo patient has an appointment

8. STORED PROCEDURES, FUNCTIONS AND TRIGGERS

8.1 STORED PROCEDURES OR FUNCTIONS

generate_bill(p_visit_id, p_consultation_fee, p_tax_rate) computes the medicine total as SUM(unit_price × duration_days) over the visit's prescription items, adds the consultation fee, applies the tax rate and inserts the bill. It is called by the Receptionist's "Generate a bill" form (POST /api/billing).

```
CREATE PROCEDURE generate_bill(
    IN p_visit_id INT,
    IN p_consultation_fee DECIMAL(10,2),
    IN p_tax_rate DECIMAL(5,4)
)
BEGIN
    DECLARE v_medicine_total DECIMAL(10,2) DEFAULT 0;
    DECLARE v_tax DECIMAL(10,2);
    DECLARE v_total DECIMAL(10,2);

    SELECT COALESCE(SUM(m.unit_price * pi.duration_days), 0)
    INTO v_medicine_total
    FROM prescription p
    JOIN prescription_item pi ON pi.prescription_id = p.prescription_id
    JOIN medicine m ON m.medicine_id = pi.medicine_id
    WHERE p.visit_id = p_visit_id;

    SET v_tax = (p_consultation_fee + v_medicine_total) * p_tax_rate;
    SET v_total = p_consultation_fee + v_medicine_total + v_tax;

    INSERT INTO bill (visit_id, consultation_fee, medicine_total, tax, total_amount, payment_status)
    VALUES (p_visit_id, p_consultation_fee, v_medicine_total, v_tax, v_total, 'pending');

    SELECT LAST_INSERT_ID() AS new_bill_id;
END$$

DELIMITER ;
```

```
pes2ug24cs918 [medirecord] >SHOW CREATE PROCEDURE generate_bill\G
*************************** 1. row ***************************
           Procedure: generate_bill
            sql_mode: ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,ERROR_FOR_DIVISION_BY_ZERO,NO_ENGINE_SUBSTITUTION
    Create Procedure: CREATE DEFINER=`root`@`localhost` PROCEDURE `generate_bill`(
    IN p_visit_id INT,
    IN p_consultation_fee DECIMAL(10,2),
    IN p_tax_rate DECIMAL(5,4)
)
BEGIN
    DECLARE v_medicine_total DECIMAL(10,2) DEFAULT 0;
    DECLARE v_tax DECIMAL(10,2);
    DECLARE v_total DECIMAL(10,2);

    SELECT COALESCE(SUM(m.unit_price * pi.duration_days), 0)
    INTO v_medicine_total
    FROM prescription p
    JOIN prescription_item pi ON pi.prescription_id = p.prescription_id
    JOIN medicine m ON m.medicine_id = pi.medicine_id
    WHERE p.visit_id = p_visit_id;

    SET v_tax = (p_consultation_fee + v_medicine_total) * p_tax_rate;
    SET v_total = p_consultation_fee + v_medicine_total + v_tax;

    INSERT INTO bill (visit_id, consultation_fee, medicine_total, tax, total_amount, payment_status)
    VALUES (p_visit_id, p_consultation_fee, v_medicine_total, v_tax, v_total, 'pending');

    SELECT LAST_INSERT_ID() AS new_bill_id;
END
character_set_client: utf8mb4
collation_connection: utf8mb4_0900_ai_ci
  Database Collation: utf8mb4_0900_ai_ci
1 row in set (0.00 sec)
```

Figure 8.1: SHOW CREATE PROCEDURE generate_bill

```
pes2ug24cs918 [medirecord] >START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

pes2ug24cs918 [medirecord] >INSERT INTO appointment (patient_id, doctor_id, scheduled_at, status, created_by)
    -> VALUES (1, 1, NOW(), 'completed', 2);
Query OK, 1 row affected (0.00 sec)

pes2ug24cs918 [medirecord] >INSERT INTO visit (appointment_id, vitals_bp, vitals_temp, vitals_pulse, diagnosis_code)
    -> VALUES (LAST_INSERT_ID(), '120/80', 98.6, 72, 'J06');
Query OK, 1 row affected (0.00 sec)

pes2ug24cs918 [medirecord] >CALL generate_bill(LAST_INSERT_ID(), 500, 0.05);
+-------------+
| new_bill_id |
+-------------+
|           9 |
+-------------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

pes2ug24cs918 [medirecord] >SELECT bill_id, consultation_fee, medicine_total, tax, total_amount FROM bill ORDER BY bill_id DESC LIMIT 1;
+---------+------------------+----------------+-------+--------------+
| bill_id | consultation_fee | medicine_total | tax   | total_amount |
+---------+------------------+----------------+-------+--------------+
|       9 |           500.00 |           0.00 | 25.00 |       525.00 |
+---------+------------------+----------------+-------+--------------+
1 row in set (0.00 sec)

pes2ug24cs918 [medirecord] >ROLLBACK;
Query OK, 0 rows affected (0.00 sec)
```

Figure 8.2: Calling generate_bill inside a transaction: 500.00 + 0.00 + 25.00 tax = 525.00

```
pes2ug24cs918 [medirecord] >SELECT COUNT(*) AS bills_after_rollback FROM bill;
+----------------------+
| bills_after_rollback |
+----------------------+
|                    8 |
+----------------------+
1 row in set (0.00 sec)
```

Figure 8.3: After ROLLBACK the bill count is back to 8

Views

```
CREATE OR REPLACE VIEW doctor_dashboard_view AS
SELECT
    d.doctor_id,
    u.name AS doctor_name,
    d.specialization,
    a.appointment_id,
    p.patient_id,
    p.name AS patient_name,
    a.scheduled_at,
    a.status AS appointment_status,
    v.visit_id,
    v.diagnosis_code,
    v.mongo_note_id,
    b.total_amount,
    b.payment_status
FROM doctor d
JOIN app_user u ON u.user_id = d.user_id
JOIN appointment a ON a.doctor_id = d.doctor_id
JOIN patient p ON p.patient_id = a.patient_id
LEFT JOIN visit v ON v.appointment_id = a.appointment_id
LEFT JOIN bill b ON b.visit_id = v.visit_id;

CREATE OR REPLACE VIEW doctor_revenue_summary AS
SELECT
    d.doctor_id,
    u.name AS doctor_name,
    COUNT(DISTINCT a.appointment_id) AS total_appointments,
    COUNT(DISTINCT p.patient_id) AS unique_patients,
    COALESCE(SUM(CASE WHEN b.payment_status = 'paid' THEN b.total_amount END), 0) AS total_revenue
FROM doctor d
JOIN app_user u ON u.user_id = d.user_id
LEFT JOIN appointment a ON a.doctor_id = d.doctor_id
```

```
LEFT JOIN patient p ON p.patient_id = a.patient_id
LEFT JOIN visit v ON v.appointment_id = a.appointment_id
LEFT JOIN bill b ON b.visit_id = v.visit_id
GROUP BY d.doctor_id, u.name;
```

```
pes2ug24cs918 [medirecord] >SELECT * FROM doctor_revenue_summary;
+-----------+-------------+--------------------+-----------------+---------------+
| doctor_id | doctor_name | total_appointments | unique_patients | total_revenue |
+-----------+-------------+--------------------+-----------------+---------------+
|         1 | Priya Sharma |                  7 |               5 |       1287.30 |
|         2 | Arvind Rao   |                  4 |               3 |       3675.53 |
+-----------+-------------+--------------------+-----------------+---------------+
2 rows in set (0.01 sec)
```

Figure 8.4: SELECT from doctor_revenue_summary (revenue counts paid bills only)

8.2 TRIGGERS

Four AFTER triggers write a JSON before/after image into audit_log, so no application code path can skip the audit trail.

Table 8.1: Triggers

Trigger	Event	Purpose
trg_prescription_insert	AFTER INSERT on prescription	Log new prescription
trg_prescription_update	AFTER UPDATE on prescription	Log change
trg_prescription_delete	AFTER DELETE on prescription	Log deletion
trg_visit_update	AFTER UPDATE on visit	Log diagnosis/vitals corrections

```
CREATE TRIGGER trg_prescription_insert
AFTER INSERT ON prescription
FOR EACH ROW
BEGIN
    INSERT INTO audit_log (table_name, record_id, action, new_value)
    VALUES ('prescription', NEW.prescription_id, 'INSERT',
            JSON_OBJECT('prescription_id', NEW.prescription_id, 'visit_id', NEW.visit_id));
END$$

DROP TRIGGER IF EXISTS trg_prescription_update$$
CREATE TRIGGER trg_prescription_update
AFTER UPDATE ON prescription
FOR EACH ROW
BEGIN
    INSERT INTO audit_log (table_name, record_id, action, old_value, new_value)
    VALUES ('prescription', NEW.prescription_id, 'UPDATE',
            JSON_OBJECT('visit_id', OLD.visit_id),
            JSON_OBJECT('visit_id', NEW.visit_id));
END$$

DROP TRIGGER IF EXISTS trg_prescription_delete$$
CREATE TRIGGER trg_prescription_delete
AFTER DELETE ON prescription
FOR EACH ROW
BEGIN
    INSERT INTO audit_log (table_name, record_id, action, old_value)
    VALUES ('prescription', OLD.prescription_id, 'DELETE',
            JSON_OBJECT('visit_id', OLD.visit_id));
END$$

DELIMITER ;

DELIMITER $$
```

```
DROP TRIGGER IF EXISTS trg_visit_update$$
CREATE TRIGGER trg_visit_update
AFTER UPDATE ON visit
FOR EACH ROW
BEGIN
    IF NOT (OLD.diagnosis_code <=> NEW.diagnosis_code) THEN
        INSERT INTO audit_log (table_name, record_id, action, old_value, new_value)
        VALUES ('visit', NEW.visit_id, 'UPDATE',
                JSON_OBJECT('diagnosis_code', OLD.diagnosis_code),
                JSON_OBJECT('diagnosis_code', NEW.diagnosis_code));
    END IF;
END$$

DELIMITER ;

DELIMITER $$
```

```
pes2ug24cs918 [medirecord] >SELECT TRIGGER_NAME, EVENT_MANIPULATION AS event, EVENT_OBJECT_TABLE AS tbl, ACTION_TIMING AS timing
    -> FROM information_schema.TRIGGERS WHERE TRIGGER_SCHEMA = 'medirecord';
+------------------------+--------+--------------+--------+
| TRIGGER_NAME           | event  | tbl          | timing |
+------------------------+--------+--------------+--------+
| trg_prescription_insert | INSERT | prescription | AFTER  |
| trg_prescription_update | UPDATE | prescription | AFTER  |
| trg_prescription_delete | DELETE | prescription | AFTER  |
| trg_visit_update       | UPDATE | visit        | AFTER  |
+------------------------+--------+--------------+--------+
4 rows in set (0.03 sec)
```

Figure 8.5: The four triggers registered in the database

```
pes2ug24cs918 [medirecord] >SHOW CREATE TRIGGER trg_visit_update\G
*************************** 1. row ***************************
             Trigger: trg_visit_update
            sql_mode: ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,ERROR_FOR_DIVISION_BY_ZERO,NO_ENGINE_SUBSTITUTION
SQL Original Statement: CREATE DEFINER=`root`@`localhost` TRIGGER `trg_visit_update` AFTER UPDATE ON `visit` FOR EACH ROW BEGIN
    IF NOT (OLD.diagnosis_code <=> NEW.diagnosis_code) THEN
        INSERT INTO audit_log (table_name, record_id, action, old_value, new_value)
        VALUES ('visit', NEW.visit_id, 'UPDATE',
                JSON_OBJECT('diagnosis_code', OLD.diagnosis_code),
                JSON_OBJECT('diagnosis_code', NEW.diagnosis_code));
    END IF;
END
  character_set_client: utf8mb4
  collation_connection: utf8mb4_0900_ai_ci
    Database Collation: utf8mb4_0900_ai_ci
               Created: 2026-10-22 23:54:45.12
1 row in set (0.00 sec)
```

Figure 8.6: SHOW CREATE TRIGGER trg_visit_update

```
pes2ug24cs918 [medirecord] >START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

pes2ug24cs918 [medirecord] >UPDATE visit SET diagnosis_code = 'J45' WHERE visit_id = 5;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

pes2ug24cs918 [medirecord] >SELECT audit_id, table_name, action FROM audit_log ORDER BY audit_id DESC LIMIT 2;
+----------+--------------+--------+
| audit_id | table_name   | action |
+----------+--------------+--------+
|       10 | visit        | UPDATE |
|        9 | prescription | INSERT |
+----------+--------------+--------+
2 rows in set (0.00 sec)

pes2ug24cs918 [medirecord] >ROLLBACK;
Query OK, 0 rows affected (0.01 sec)
```

Figure 8.7: Trigger fires on UPDATE of visit: audit_log row 10 created (then rolled back)

```
pes2ug24cs918 [medirecord] >SELECT audit_id, table_name, action, changed_at FROM audit_log;
+----------+--------------+--------+---------------------+
| audit_id | table_name   | action | changed_at          |
+----------+--------------+--------+---------------------+
|        1 | prescription | INSERT | 2026-11-02 22:32:01 |
|        2 | prescription | INSERT | 2026-11-02 22:32:01 |
|        3 | prescription | INSERT | 2026-11-02 22:32:01 |
|        4 | prescription | INSERT | 2026-11-02 22:32:01 |
|        5 | visit        | UPDATE | 2026-11-02 22:32:01 |
|        6 | prescription | INSERT | 2026-11-02 22:32:01 |
|        7 | prescription | INSERT | 2026-11-02 22:32:01 |
|        8 | prescription | INSERT | 2026-11-02 22:32:01 |
|        9 | prescription | INSERT | 2026-11-02 22:32:01 |
+----------+--------------+--------+---------------------+
9 rows in set (0.02 sec)
```

Figure 8.8: audit_log contents after seeding

Integrity constraints in action

```
C:\Users\Rakshitha\Desktop\MediRecord>mysql -u root -p --force medirecord < sql\demo_integrity_errors.sql
Enter password: *******
expected_to_fail
1. CHECK: invalid gender
ERROR 3819 (HY000) at line 16: Check constraint 'patient_chk_1' is violated.
expected_to_fail
2. CHECK: negative medicine price
ERROR 3819 (HY000) at line 20: Check constraint 'medicine_chk_2' is violated.
expected_to_fail
3. UNIQUE: duplicate login email
ERROR 1062 (23000) at line 24: Duplicate entry 'admin@medirecord.com' for key 'app_user.email'
expected_to_fail
4. UNIQUE: second bill for the same visit
ERROR 1062 (23000) at line 29: Duplicate entry '1' for key 'bill.visit_id'
expected_to_fail
5. FOREIGN KEY: appointment for non-existent patient
ERROR 1452 (23000) at line 33: Cannot add or update a child row: a foreign key constraint fails (`medirecord`.`appointme
nt`, CONSTRAINT `appointment_ibfk_1` FOREIGN KEY (`patient_id`) REFERENCES `patient` (`patient_id`) ON DELETE RESTRICT)
expected_to_fail
6. RESTRICT: delete a patient with appointments
ERROR 1451 (23000) at line 38: Cannot delete or update a parent row: a foreign key constraint fails (`medirecord`.`appoi
ntment`, CONSTRAINT `appointment_ibfk_1` FOREIGN KEY (`patient_id`) REFERENCES `patient` (`patient_id`) ON DELETE RESTRI
CT)
expected_to_fail
7. CHECK: invalid appointment status
ERROR 3819 (HY000) at line 42: Check constraint 'appointment_chk_1' is violated.
result
Done. Every statement above was rejected; data unchanged.
```

Figure 8.9: The seven statements the database rejects (CHECK, UNIQUE, FOREIGN KEY, RESTRICT)

```
pes2ug24cs918 [medirecord] >INSERT INTO patient (name, dob, gender, phone) VALUES ('X', '2000-01-01', 'Alien', '1');
ERROR 3819 (HY000): Check constraint 'patient_chk_1' is violated.
pes2ug24cs918 [medirecord] >INSERT INTO medicine (name, form, unit_price, stock_qty) VALUES ('Bad Price', 'tablet', -5, 10);
ERROR 3819 (HY000): Check constraint 'medicine_chk_2' is violated.
pes2ug24cs918 [medirecord] >INSERT INTO app_user (name, email, password_hash, role_id) VALUES ('Dup', 'admin@medirecord.com', 'x', 1);
ERROR 1062 (23000): Duplicate entry 'admin@medirecord.com' for key 'app_user.email'
pes2ug24cs918 [medirecord] >INSERT INTO bill (visit_id, consultation_fee, medicine_total, tax, total_amount) VALUES (1, 100, 0, 0, 100);
ERROR 1062 (23000): Duplicate entry '1' for key 'bill.visit_id'
pes2ug24cs918 [medirecord] >DELETE FROM patient WHERE patient_id = 1;
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails (`medirecord`.`appointment`, CONSTRAINT `appointment_ibfk_1
` FOREIGN KEY (`patient_id`) REFERENCES `patient` (`patient_id`) ON DELETE RESTRICT)
pes2ug24cs918 [medirecord] >
```

Figure 8.10: Typed integrity violations in the MySQL client

Indexes and security

```
pes2ug24cs918 [medirecord] >EXPLAIN SELECT appointment_id FROM appointment WHERE doctor_id = 1\G
*************************** 1. row ***************************
           id: 1
  select_type: SIMPLE
        table: appointment
   partitions: NULL
         type: ref
possible_keys: idx_appointment_doctor
          key: idx_appointment_doctor
      key_len: 4
          ref: const
         rows: 7
     filtered: 100.00
        Extra: Using index
1 row in set, 1 warning (0.00 sec)
```

Figure 8.11: EXPLAIN: the appointment lookup by doctor uses idx_appointment_doctor (type ref)

```
pes2ug24cs918 [medirecord] >SELECT email, LENGTH(password_hash) AS hash_length, LEFT(password_hash, 7) AS prefix FROM app_user;
+---------------------------+-------------+---------+
| email                     | hash_length | prefix  |
+---------------------------+-------------+---------+
| admin@medirecord.com      |          60 | $2b$10$ |
| reception@medirecord.com  |          60 | $2b$10$ |
| priya.sharma@medirecord.com |          60 | $2b$10$ |
| arvind.rao@medirecord.com |          60 | $2b$10$ |
+---------------------------+-------------+---------+
4 rows in set (0.00 sec)
```

Figure 8.12: Passwords are bcrypt hashes (60 characters, prefix $2b$10$)

8.3 NOSQL (MONGODB) IMPLEMENTATION

Free-text clinical notes and symptom tags are stored in the MongoDB collection clinical_notes, one document per visit, linked to MySQL through visit_id. A text index on notes and symptoms enables ranked full-text search; a unique index on visit_id keeps one note per visit; further indexes support filtering by doctor and diagnosis.

```
{ visit_id, patient_id, doctor_id, diagnosis_code,
  symptoms: [ "chest tightness", "wheezing", ... ],
  notes: "free text", created_at }

db.clinical_notes.createIndex({ notes: "text", symptoms: "text" }, { name: "idx_notes_text_search"
})
db.clinical_notes.createIndex({ visit_id: 1 }, { unique: true })
```

```
medirecord> db.clinical_notes.countDocuments()
8
medirecord> db.clinical_notes.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  {
    v: 2,
    key: { _fts: 'text', _ftsx: 1 },
    name: 'idx_notes_text_search',
    weights: { notes: 1, symptoms: 1 },
    default_language: 'english',
    language_override: 'language',
    textIndexVersion: 3
  },
  { v: 2, key: { visit_id: 1 }, name: 'visit_id_1', unique: true },
  { v: 2, key: { doctor_id: 1 }, name: 'doctor_id_1' },
  { v: 2, key: { diagnosis_code: 1 }, name: 'diagnosis_code_1' }
]
medirecord>
```

Figure 8.13: clinical_notes: 8 documents and its indexes (text index, unique visit_id, doctor_id, diagnosis_code)

```
medirecord> db.clinical_notes.findOne({}, {_id:0})
{
  visit_id: 1,
  patient_id: 1,
  doctor_id: 2,
  diagnosis_code: 'J45',
  symptoms: [ 'chest tightness', 'wheezing', 'shortness of breath' ],
  notes: 'Patient reports wheezing at night and chest tightness on exertion. Known asthma since childhood. Reduced peak
flow. Advised to avoid dust exposure.',
  created_at: ISODate('2026-11-02T17:02:01.618Z')
}
medirecord>
```

Figure 8.14: Data model: one clinical_notes document

```
medirecord> db.clinical_notes.find({ $text: { $search: "chest tightness wheezing" } },
{ _id:0, visit_id:1, symptoms:1, diagnosis_code:1, score:{ $meta:"textScore" } }).sort(
{ score:{ $meta:"textScore" } })
[
  {
    visit_id: 1,
    diagnosis_code: 'J45',
    symptoms: [ 'chest tightness', 'wheezing', 'shortness of breath' ],
    score: 4.083333333333333
  },
  {
    visit_id: 2,
    diagnosis_code: 'J45',
    symptoms: [ 'wheezing', 'dry cough', 'chest tightness' ],
    score: 3.0357142857142856
  },
  {
    visit_id: 5,
    diagnosis_code: 'J44',
    symptoms: [ 'shortness of breath', 'chronic cough', 'wheezing' ],
    score: 1.5333333333333332
  }
]
medirecord>
```

Figure 8.15: Ranked full-text search with $text and textScore

```
medirecord> db.clinical_notes.find({ diagnosis_code: "J45" }, { _id:0, visit_id:1, doct
doctor_id:1, symptoms:1 })
[
  {
    visit_id: 1,
    doctor_id: 2,
    symptoms: [ 'chest tightness', 'wheezing', 'shortness of breath' ]
  },
  {
    visit_id: 2,
    doctor_id: 2,
    symptoms: [ 'wheezing', 'dry cough', 'chest tightness' ]
  }
]
```

Figure 8.16: Filtering by diagnosis code J45

```
medirecord> db.clinical_notes.aggregate([ { $group: { _id: "$doctor_id", notes: { $sum: 1 } } },
{ $sort: { _id: 1 } } ])
[ { _id: 1, notes: 5 }, { _id: 2, notes: 3 } ]
medirecord> db.clinical_notes.aggregate([ { $group: { _id: "$doctor_id", notes: { $sum: 1 } } },
{ $sort: { _id: 1 } } ])
[ { _id: 1, notes: 5 }, { _id: 2, notes: 3 } ]
medirecord> db.clinical_notes.aggregate([ { $unwind: "$symptoms" }, { $group: { _id: { diagnosis_
code: "$diagnosis_code", symptom: "$symptoms" }, count: { $sum: 1 } } }, { $sort: { count: -1 } }
, { $limit: 8 } ])
[
  { _id: { diagnosis_code: 'J06', symptom: 'fever' }, count: 2 },
  { _id: { diagnosis_code: 'J45', symptom: 'wheezing' }, count: 2 },
  { _id: { diagnosis_code: 'J06', symptom: 'sore throat' }, count: 2 },
  {
    _id: { diagnosis_code: 'J45', symptom: 'chest tightness' },
    count: 2
  },
  { _id: { diagnosis_code: 'K29', symptom: 'abdominal pain' }, count: 1 },
  {
    _id: { diagnosis_code: 'G43', symptom: 'sensitivity to light' },
    count: 1
  },
  { _id: { diagnosis_code: 'J44', symptom: 'wheezing' }, count: 1 },
  { _id: { diagnosis_code: 'G43', symptom: 'headache' }, count: 1 }
]
medirecord>
```

Figure 8.17: Aggregation: notes per doctor and symptoms per diagnosis

9. FRONT END DEVELOPMENT

The front end is a Next.js (App Router) application with three role-specific dashboards. Route access is enforced in proxy.js, and every API route re-checks the role with requireRole(). All database access uses parameterised queries. The screenshots show the running application with the demo data.

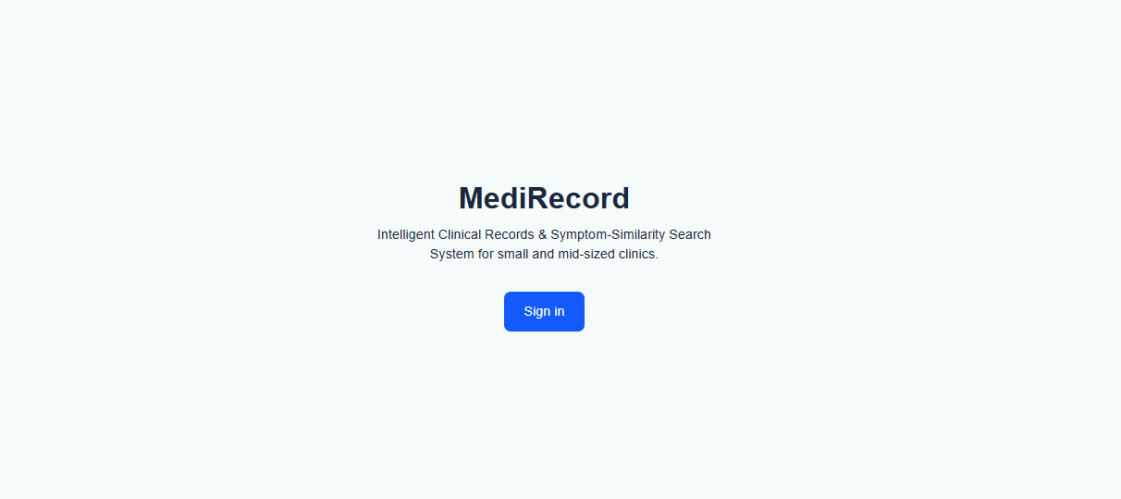


Figure 9.1: Home page with Sign in

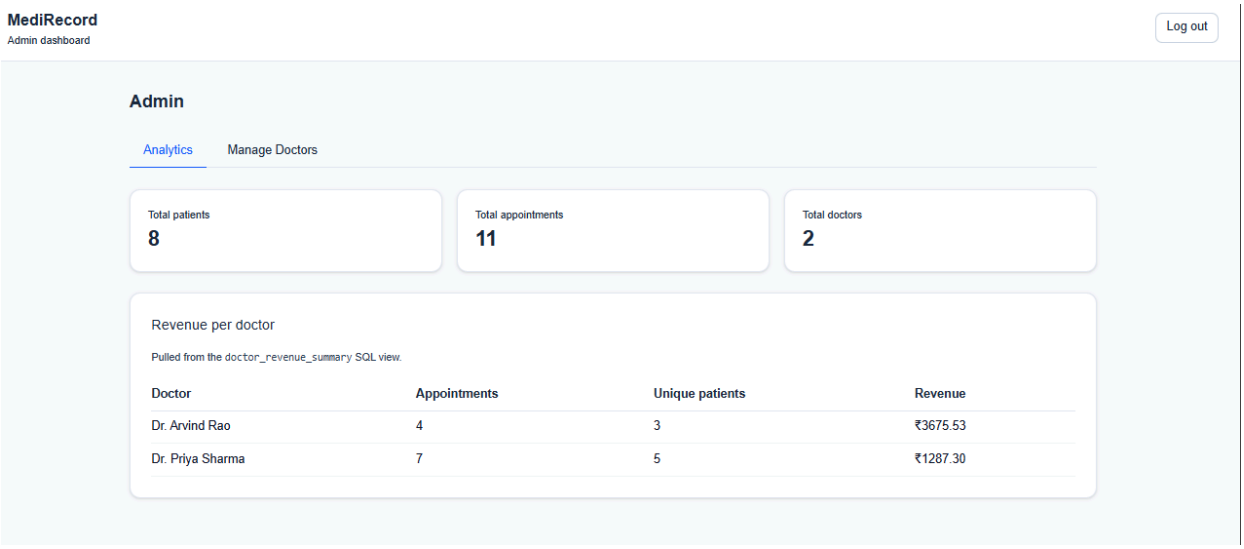


Figure 9.2: Admin – analytics (revenue per doctor from the SQL view)

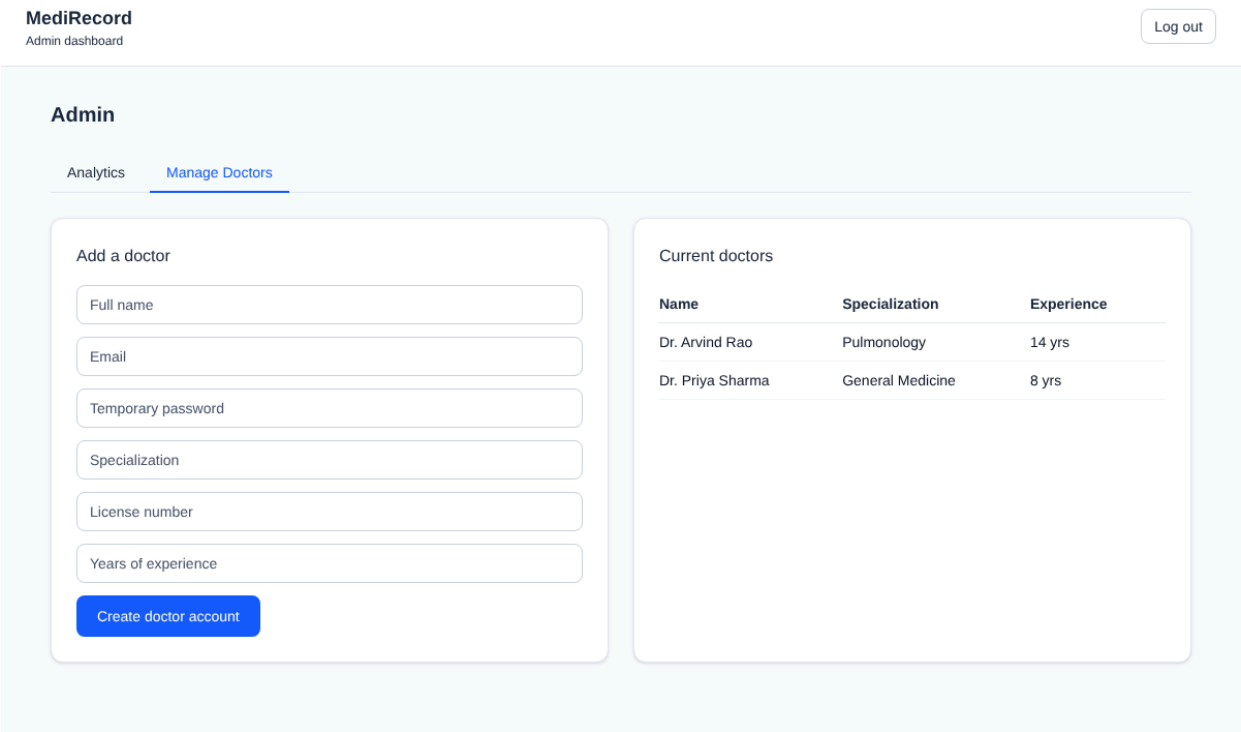


Figure 9.3: Admin – manage doctors

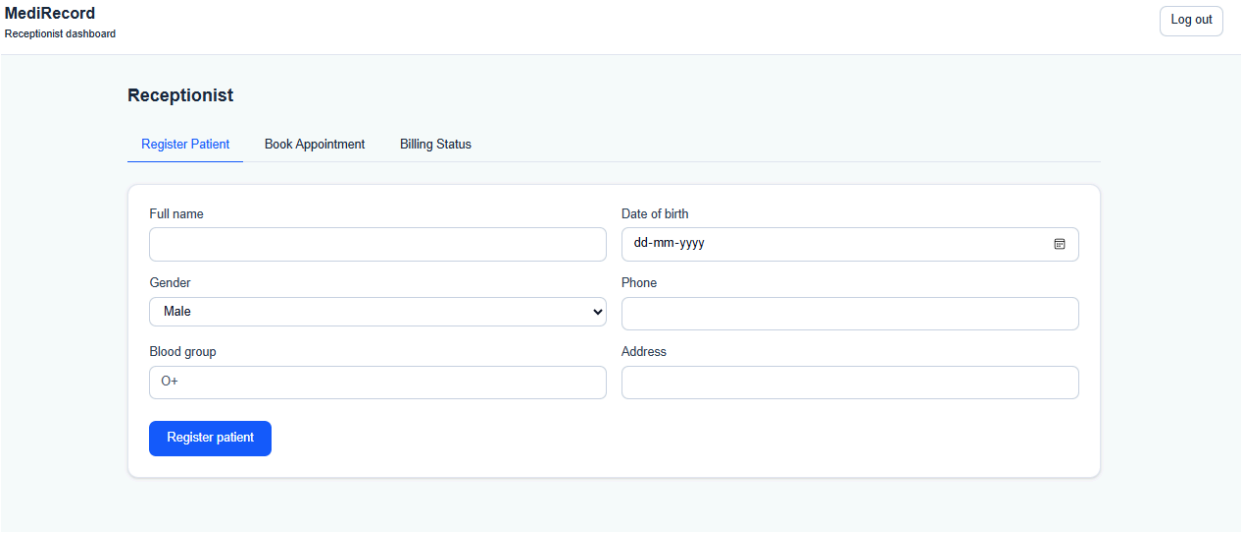


Figure 9.4: Receptionist – register patient

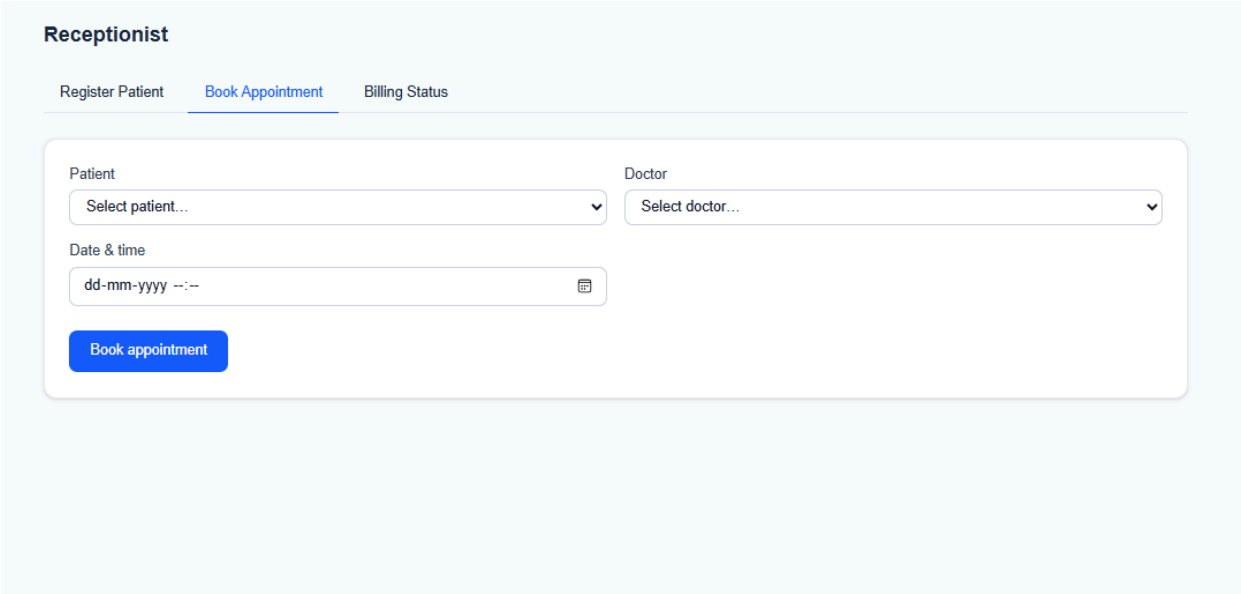
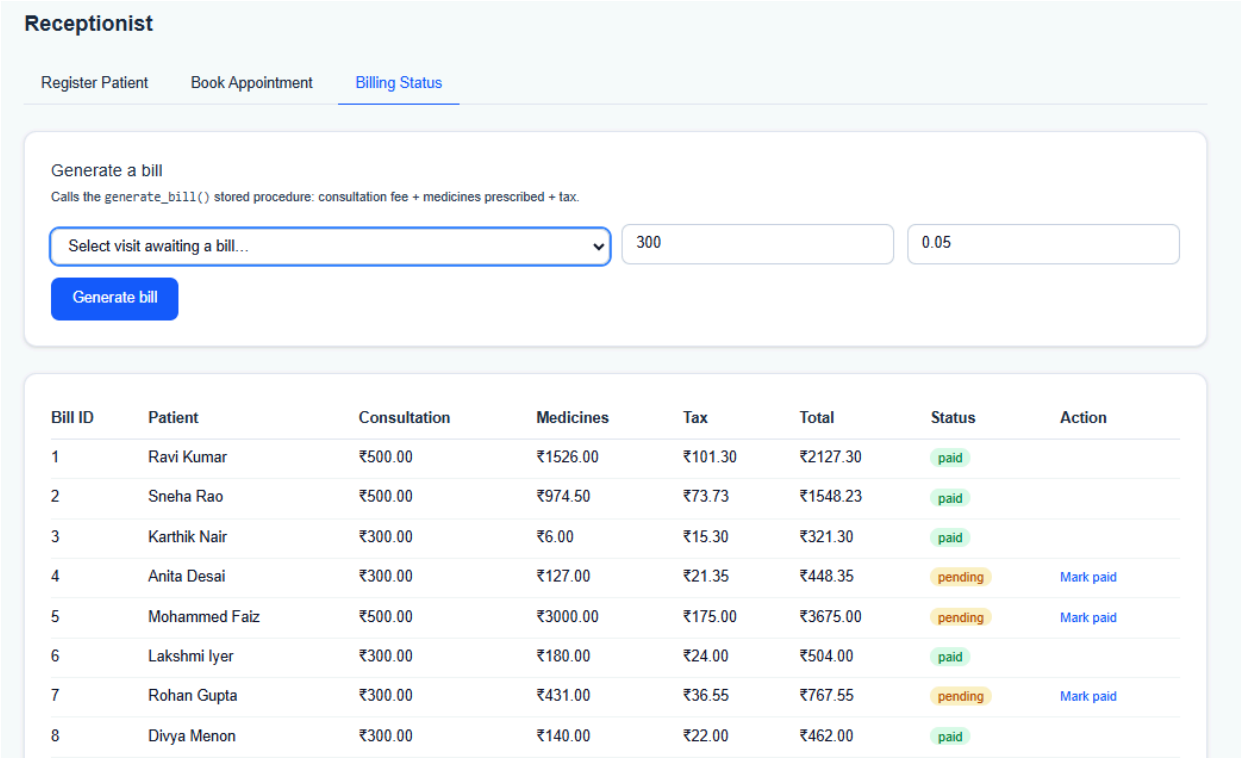


Figure 9.5: Receptionist – book appointment



Bill ID	Patient	Consultation	Medicines	Tax	Total	Status	Action
1	Ravi Kumar	₹500.00	₹1526.00	₹101.30	₹2127.30	paid	
2	Sneha Rao	₹500.00	₹974.50	₹73.73	₹1548.23	paid	
3	Karthik Nair	₹300.00	₹6.00	₹15.30	₹321.30	paid	
4	Anita Desai	₹300.00	₹127.00	₹21.35	₹448.35	pending	Mark paid
5	Mohammed Faiz	₹500.00	₹3000.00	₹175.00	₹3675.00	pending	Mark paid
6	Lakshmi Iyer	₹300.00	₹180.00	₹24.00	₹504.00	paid	
7	Rohan Gupta	₹300.00	₹431.00	₹36.55	₹767.55	pending	Mark paid
8	Divya Menon	₹300.00	₹140.00	₹22.00	₹462.00	paid	

Figure 9.6: Receptionist – generate bill (stored procedure) and bill list

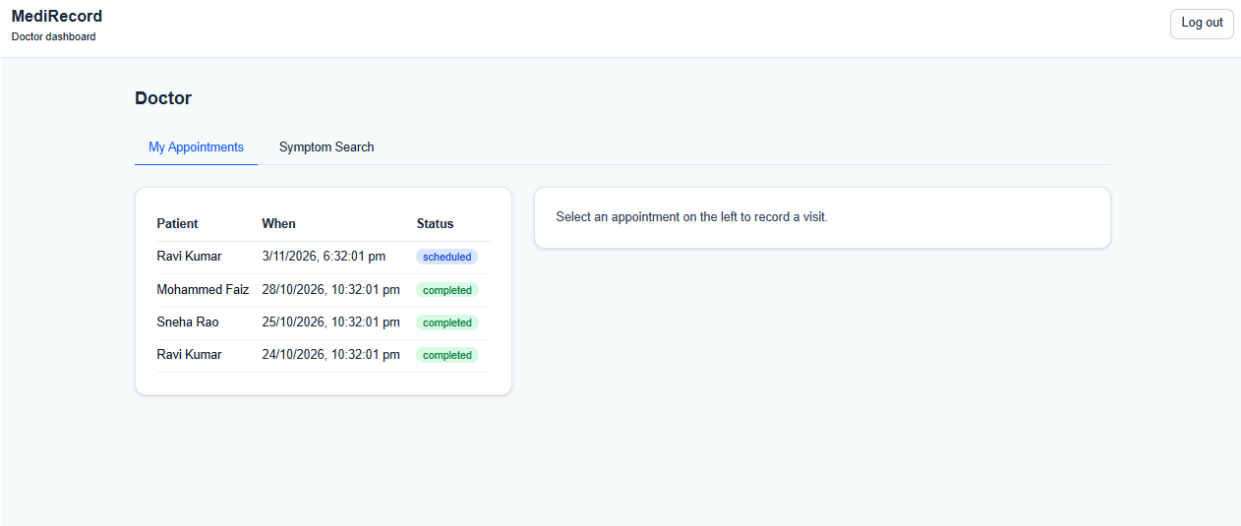


Figure 9.7: Doctor – my appointments

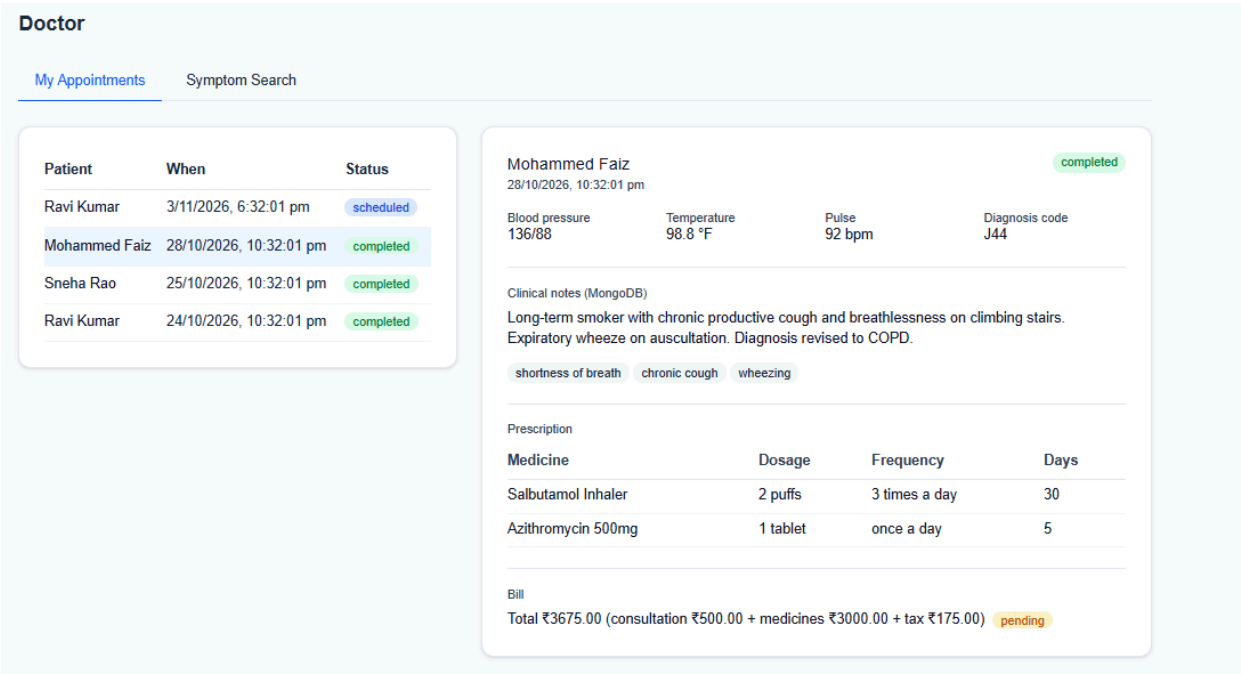


Figure 9.8: Doctor – completed visit: vitals and diagnosis (MySQL), notes and symptoms (MongoDB), prescription and bill

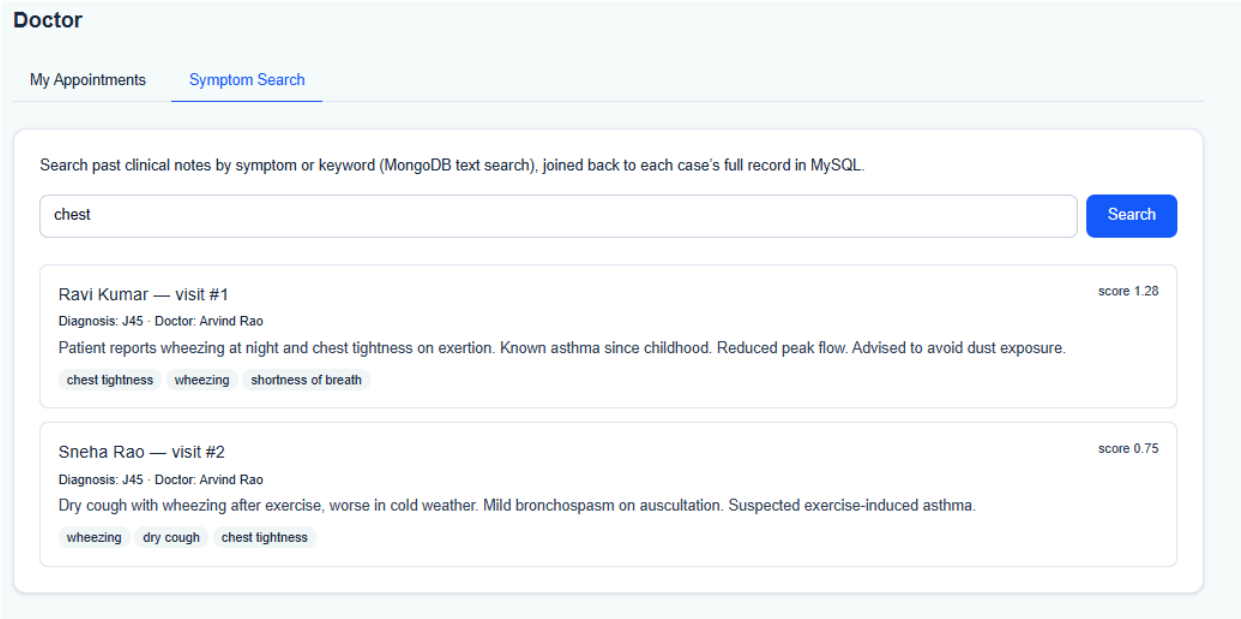


Figure 9.9: Doctor – Symptom Search: ranked similar cases from MongoDB joined with SQL data

Table 9.1: Demo logins (seeded)

Role	Email	Password
Admin	admin@medirecord.com	rkb@123
Receptionist	reception@medirecord.com	password123
Doctor	arvind.rao@medirecord.com	doctor@123
Doctor	priya.sharma@medirecord.com	doctor@123

10. TESTING

Test cases were executed against the MySQL database, the MongoDB collection and the running application. T9 (saving a visit from the form) is still to be confirmed and is highlighted.

Table 10.1: Test cases

ID	Test	Expected	Actual	Result
T1	Insert patient with gender ‘Alien’	Rejected (CHECK)	Rejected, error 3819	Passed
T2	Insert medicine with negative price	Rejected (CHECK)	Rejected, CHECK violation	Passed
T3	Second bill for the same visit	Rejected (UNIQUE)	Rejected	Passed
T4	Delete patient who has appointments	Rejected (FK RESTRICT)	Rejected	Passed
T5	Update visit diagnosis	Audit row via trigger	audit_log row written	Passed
T6	CALL generate_bill	Bill = fee + medicines + tax	Totals correct	Passed
T7	Wrong role opens /admin	Redirected / 401	Redirected	Passed
T8	Receptionist generates bill from UI	Bill appears, status pending	Bill created	Passed
T9	Doctor saves visit (MySQL + MongoDB)	Visit and note saved	Saved visit	Passed
T10	Symptom search ‘chest’	Ranked similar cases	Ravi Kumar (score 1.28), Sneha Rao (0.75)	Passed
T11	Completed visit shows notes + symptoms	Notes displayed	Notes, symptom chips, prescription and bill shown	Passed

11. CONCLUSION

MediRecord demonstrates how a relational database and a document database can complement each other in one application: MySQL guarantees integrity for structured clinical and billing data through constraints, triggers, a stored procedure, views and transactions, while MongoDB gives flexible storage and ranked full-text search for free-text notes. The role-based web application ties both together securely.

Future work

- Deployment to the cloud (hosted MySQL, MongoDB Atlas) for reliability.
- Patient portal, SMS/e-mail reminders and payment gateway integration.
- Stronger similarity search (synonyms, vector embeddings).

REFERENCES

[1] MySQL 8.0 Reference Manual, Oracle Corporation. https://dev.mysql.com/doc/refman/8.0/en/

[2] MongoDB Manual – Text Search and Indexes. https://www.mongodb.com/docs/manual/

[3] Next.js Documentation. https://nextjs.org/docs

[4] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, Pearson.

[5] Course material – DBMS Experiential Learning, PES University.

GitHub Link: https://github.com/rakshithakb5/MediRecord.git

APPENDIX A DEFINITIONS, ACRONYMS AND ABBREVIATIONS

Term	Meaning
DBMS	Database Management System
SQL / NoSQL	Structured Query Language / non-relational database
DDL / DML	Data Definition / Data Manipulation Language
ER	Entity–Relationship
PK / FK	Primary Key / Foreign Key
JWT	JSON Web Token
bcrypt	Adaptive password-hashing function
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
ICD	International Classification of Diseases (diagnosis codes such as J45)